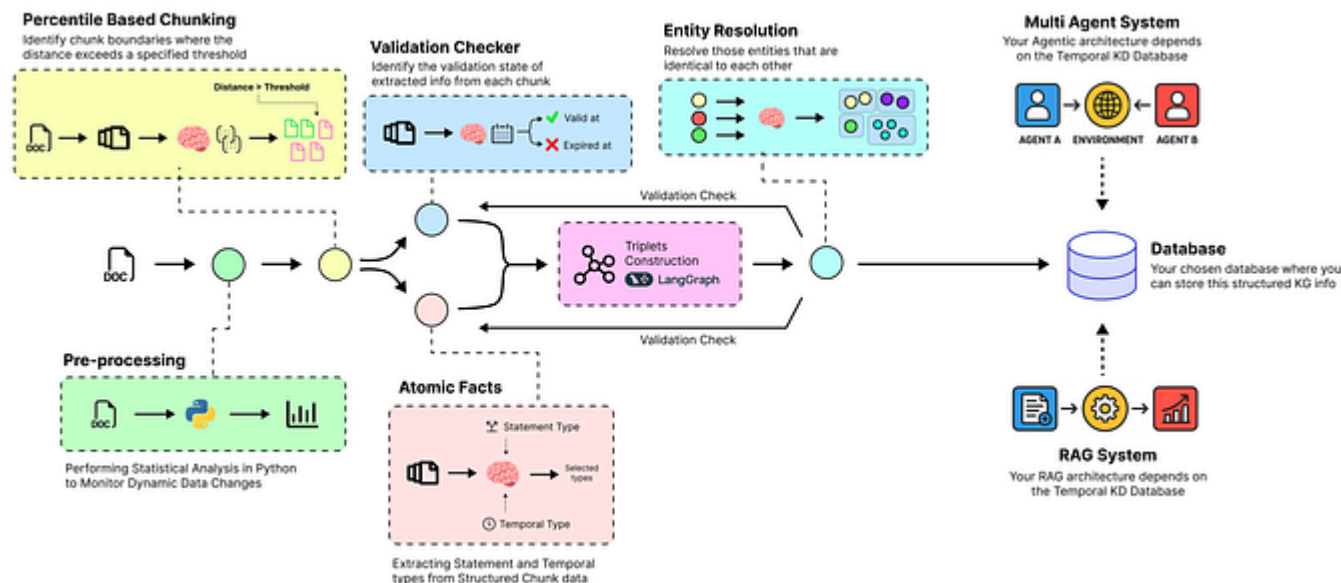


[< Go to the original](#)

Building a Temporal AI Agent to Optimize Evolving Knowledge Bases in Modern RAG Systems

Atomic Facts, Validation, Entity Resolution, KG and more



Fareed Khan

Read this story for free: [link](#)

RAG or agentic architectures for answering questions depends on a dynamic knowledge base that keeps updating over time, such as financial reports or documentation, so that the reasoning and planning steps remains logical and accurate.

To handle such a knowledge base, where the size continuously grows and the chances of hallucinations can increase, **a separate logical-temporal (time-aware) agentic pipeline is required to manage this evolving knowledge base** within your AI product. This pipeline includes ...

1. **Semantic Chunking:** Breaks down large, raw documents into small, contextually meaningful text chunks.
2. **Atomic Facts:** Uses an LLM to read each chunk and extract atomic facts, their timestamps, and the entities involved.
3. **Entity Resolution:** Cleans the data by automatically finding and merging duplicate entities (e.g., "AMD" and "Advanced Micro Devices").

5. **Knowledge Graph Construction:** Assembles the final, clean, time-stamped facts into a connected graph structure that our AI agent can query.
6. **Optimized Knowledge Base:** Stores the final, dynamic knowledge graph in a scalable cloud database, creating the reliable, up-to-date "brain" on top of which the final RAG or agentic system is built.

In this blog we are going to create ...

An end-to-end temporal agentic pipeline that transforms raw data into a dynamic knowledge base, and then build a multi-agent system on top of it to measure its performance.

All (Theory + Notebook) Step-by-step implementation is available in my GitHub Repo:

Dynamic Knowledge Base Using AI...

Optimizing Dynamic Knowledge Base Using AI Agent. Contribute to FareedKhan-dev/temporal-ai-agent-pipeline development...

github.com

Table of Contents

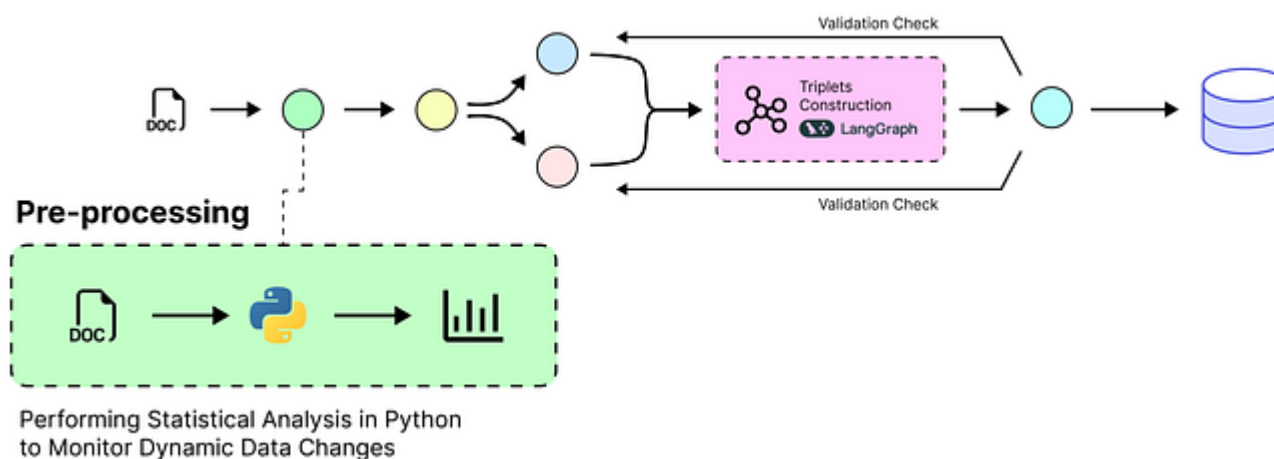
- [Pre-processing and Analyzing our Dynamic Data](#)
- [Percentile Semantic Chunking](#)
- [Extracting Atomic Facts with a Statement Agent](#)
- [Pinpointing Time with a Validation Check Agent](#)
- [Structuring Facts into Triplets](#)
- [Assembling the Temporal Event](#)
- [Automating the Pipeline with LangGraph](#)
- [Cleaning Our Data with Entity Resolution](#)
- [Making Our Knowledge Dynamic with an Invalidation Agent](#)
- [Assembling the Temporal Knowledge Graph](#)

What Can We Do Next?

Pre-processing and Analyzing our Dynamic Data

We will be working with datasets that evolve continuously over time and ...

The financial situation of a company is one of the best examples.



Pre-processing Step (Created by [Fareed Khan](#))

Companies regularly share updates on their financial performance, such as stock price movements, major developments like changes in executive leadership, and

In the medical domain, ICD coding is another example of evolving data, where the transition from ICD-9 to ICD-10 increased diagnosis codes from about 14,000 to 68,000.

To mimic a real world scenario, We will be working with earnings_call huggingface dataset from John Henning. It contains information about the financial performance of different companies over a time frame.

Let's load this dataset and perform some statistical analysis on it to get used to it.

Copy

```
# Import loader for Hugging Face datasets
from langchain_community.document_loaders import HuggingFaceDatasetLoader

# Dataset configuration
hf_dataset_name = "jllh-ibm/earnings_call" # HF dataset name
subset_name = "transcripts" # Dataset subset to load

# Create the loader (defaults to 'train' split)
loader = HuggingFaceDatasetLoader(
    path=hf_dataset_name,
    name=subset_name,
    page_content_column="transcript" # Column containing the main text
```

```
# THIS IS THE KEY STEP: THE LOADER PROCESSES THE DATASET AND RETURNS A LIST OF  
documents = loader.load()
```

We are focusing on the *transcript* subset of this dataset, which contains the raw textual information about different companies. This is the basic structure of the dataset and serves as the starting point for any RAG or AI Agent architecture.

Copy

```
# Let's inspect the result to see the difference  
print(f"Loaded {len(documents)} documents.")  
  
#### OUTPUT ####  
Loaded 188 documents.
```

There are a total of 188 transcripts in our data. These transcripts belong to different companies, and we need to count how many unique companies are represented in our dataset.

Copy

```
# Count how many documents each company has  
company_counts = {}
```

```
for doc in documents:
    company = doc.metadata.get("company") # Extract company from metadata
    if company:
        company_counts[company] = company_counts.get(company, 0) + 1

# Display the counts
print("Total company counts:")
for company, count in company_counts.items():
    print(f" - {company}: {count}")

#### OUTPUT ####
Total company counts:
- AMD: 19
- AAPL: 19
- INTC: 19
- MU: 17
- GOOGL: 19
- ASML: 19
- CSCO: 19
- NVDA: 19
- AMZN: 19
- MSFT: 19
```

Almost all companies have equal distribution ratios. Take a look at the metadata of a random transcript.


```
# Print metadata for two sample documents (index 0 and 33)
print("Metadata for document[0]:")
print(documents[0].metadata)

print("\nMetadata for document[33]:")
print(documents[33].metadata)

#### OUTPUT ####
{'company': 'AMD', 'date': datetime.date(2016, 7, 21)}
{'company': 'AMZN', 'date': datetime.date(2019, 10, 24)}
```

The **company** field simply indicates which company the transcript belongs to, and the **date** field represents the timeframe the information is based on.

Copy

```
# Print the first 200 characters of the first document's content
first_doc = documents[0]
print(first_doc.page_content[:200])

#### OUTPUT ####
Thomson Reuters StreetEvents Event Transcript
E D I T E D   V E R S I O N
Q2 2016 Advanced Micro Devices Inc Earnings Call
JULY 21, 2016 / 9:00PM GMT
```

By printing a sample of our document, we can get a high-level overview. For example, the current sample shows the quarterly report of AMD.

Transcripts can be very long because they represent information for a given timeframe and contain a large amount of detail. We need to check how many words, on average, those 188 transcripts contain.

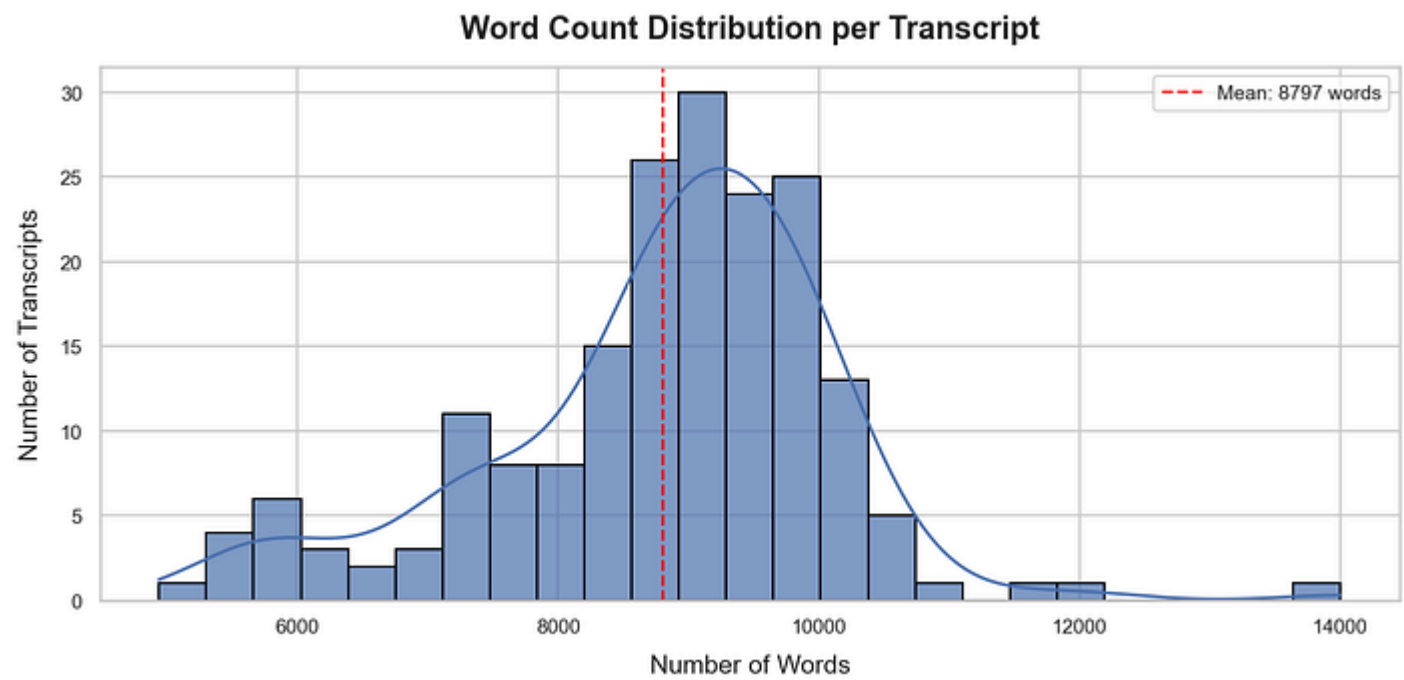
Copy

```
# Calculate the average number of words per document
total_words = sum(len(doc.page_content.split()) for doc in documents)
average_words = total_words / len(documents) if documents else 0

print(f"Average number of words in documents: {average_words:.2f}")

#### OUTPUT ####
Average number of words in documents: 8797.124
```

~9K words per transcript is quite huge, as it surely contains a large amount of information. But this is exactly what we need creating a well-structured



Words Distribution Plot (Created by [Fareed Khan](#))

Normally, financial data is based on different timeframes, each representing different information about what is happening during that period. We can extract those timeframes from the transcripts using plain Python code instead of LLMs to save costs.

Copy

```
# Helper function to extract a quarter string (e.g., "Q1 2023") from text
def find_quarter(text: str) -> str | None:
    """Return the first quarter-year match found in the text, or None if absent
    # Match pattern: 'Q' followed by 1 digit, a space, and a 4-digit year
    match = re.findall(r"Q\d\s\d{4}", text)
    return match[0] if match else None

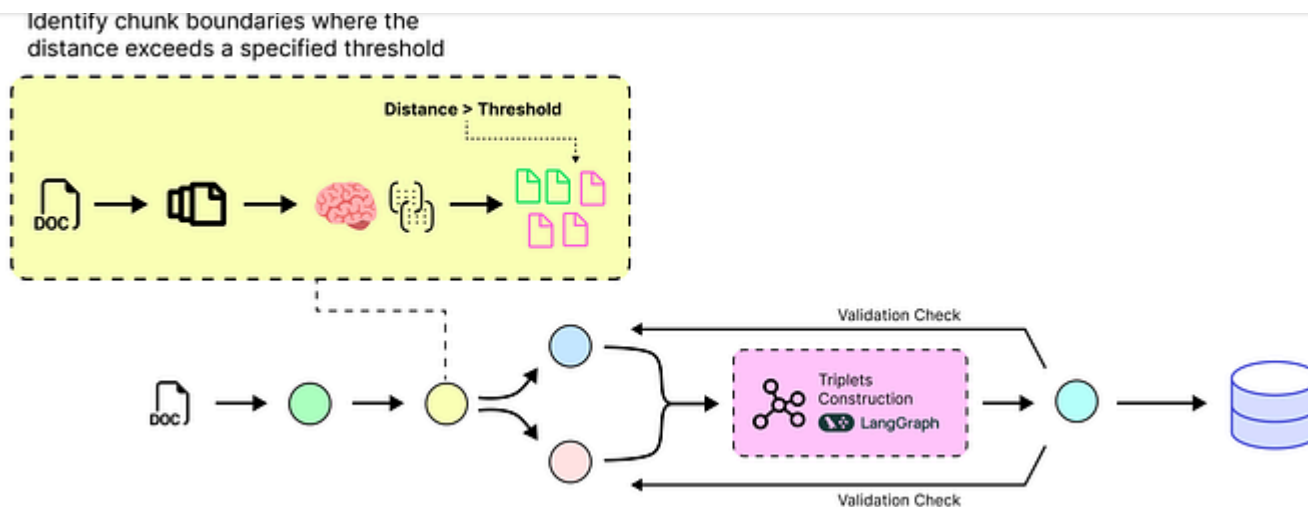
# Test on the first document
quarter = find_quarter(documents[0].page_content)
print(f"Extracted Quarter for the first document: {quarter}")

#### OUTPUT ####
Extracted Quarter for the first document: Q2 2016
```

A better way to perform quarter-date extraction is through LLMs, as they can understand the data in a deeper way. However, since our data is already well-structured in terms of text, we can proceed without them for now.

Now that we have a quick understanding of our dynamic data, we can start building the knowledge base through a temporal AI agent.

Percentile Semantic Chunking

Percentile Based Chunking (Created by [Fareed Khan](#))

Normally, we chunk the data based on either random splits or at meaningful sentence boundaries, such as ending at a full stop. However, this approach might lead to losing some information. For example:

Net income rose 12% to \$2.1M. The increase was driven by lower operating expenses

If we split at the full stop, we lose the compact connection that the increase in net income was due to lower operating expenses.

1. The document is split into sentences using a regex, usually breaking after . , ? , or ! .
2. Each sentence is converted into a high-dimensional vector using the embedding model.
3. The semantic distance between consecutive sentence vectors is calculated, with larger values indicating bigger topic changes.
4. All distances are collected and the chosen percentile, such as the 95th, is determined to capture unusually large jumps.
5. Boundaries where the distance is greater than or equal to this threshold are marked as chunk breakpoints.
6. Sentences between these boundaries are grouped into chunks, applying `min_chunk_size` to avoid overly small chunks and `buffer_size` to add overlap if needed.

Copy

```
from langchain_nebius import NebiusEmbeddings

# Set Nebius API key (⚠️ Avoid hardcoding secrets in production code)
os.environ["NEBIUS_API_KEY"] = "YOUR_API_KEY_HERE"
```

We are using Qwen3-8B to generate embeddings through Nebius AI in LangChain. Of course, there are many other embedding providers supported under the LangChain module.

Copy

```
from langchain_experimental.text_splitter import SemanticChunker

# Create a semantic chunker using percentile thresholding
langchain_semantic_chunker = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile", # Use percentile-based splitting
    breakpoint_threshold_amount=95        # split at 95th percentile
)
```

We have chosen the 95th percentile value, which means that if the distance between consecutive sentences goes above this value, it will be considered a breakpoint. Using a loop, we can simply start the chunking process on our transcripts.

Copy

```
# Printing total number of docs (188) We already know that
print(f"Processing {len(documents)} documents using LangChain's SemanticChunker")

# Chunk each transcript document
for doc in tqdm(documents, desc="Chunking Transcripts with LangChain"):
    # Extract quarter info and copy existing metadata
    quarter = find_quarter(doc.page_content)
    parent_metadata = doc.metadata.copy()
    parent_metadata["quarter"] = quarter

    # Perform semantic chunking (returns Document objects with metadata attached)
    chunks = langchain_semantic_chunker.create_documents(
        [doc.page_content],
        metadatas=[parent_metadata]
    )

    # Collect all chunks
    chunked_documents_lc.extend(chunks)

#### OUTPUT ####
Processing 188 documents using LangChain's SemanticChunker...
Chunking Transcripts with LangChain: 100%|██████████| 188/188 [01:03:44<00:00,
```

This process will take a lot of time because each transcript has around 8K words, as we saw earlier. To speed things up, we can use asynchronous functions to

Copy

```
# Analyze the results of the LangChain chunking process
original_doc_count = len(docs_to_process)
chunked_doc_count = len(chunked_documents_lc)

print(f"Original number of documents (transcripts): {original_doc_count}")
print(f"Number of new documents (chunks): {chunked_doc_count}")
print(f"Average chunks per transcript: {chunked_doc_count / original_doc_count}")

#### OUTPUT ####
Original number of documents (transcripts): 188
Number of new documents (chunks): 3556
Average chunks per transcript: 19.00
```

We have, on average, 19 chunks per transcript. Let's inspect a random chunk from one of our transcripts.

Copy

```
# Inspect the 11th chunk (index 10)
sample_chunk = chunked_documents_lc[10]
print("Sample Chunk Content (first 30 chars):")
print(sample_chunk.page_content[:30] + "...")
```

```
print(sample_chunk.metadata)

# Calculate average word count per chunk
total_chunk_words = sum(len(doc.page_content.split()) for doc in chunked_documents)
average_chunk_words = total_chunk_words / chunked_doc_count if chunked_documents else 0

print(f"\nAverage number of words per chunk: {average_chunk_words:.2f}")

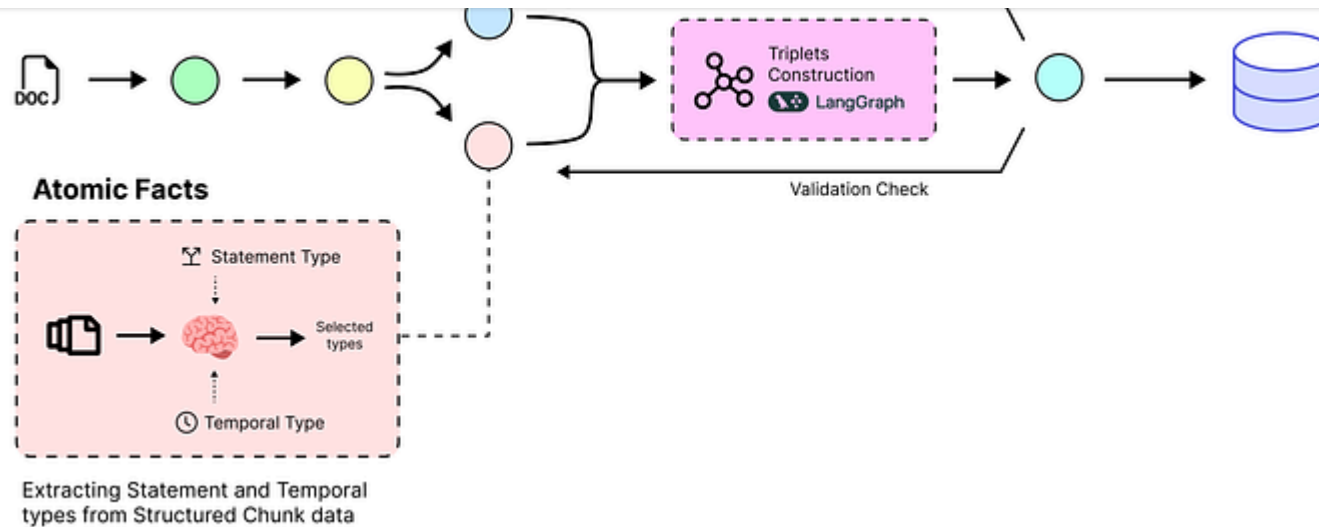
#### OUTPUT ####
Sample Chunk Content (first 30 chars):
No, that is a fair question, Matt. So we have been very focused ...

Sample Chunk Metadata:
{'company': 'AMD', 'date': datetime.date(2016, 7, 21), 'quarter': 'Q2 2016'}
Average number of words per chunk: 445.42
```

Our chunk data metadata has changed slightly, including some additional information like the quarter the chunk belongs to, along with the datetime in a Python-friendly format for easy retrieval.

Extracting Atomic Facts with a Statement Agent

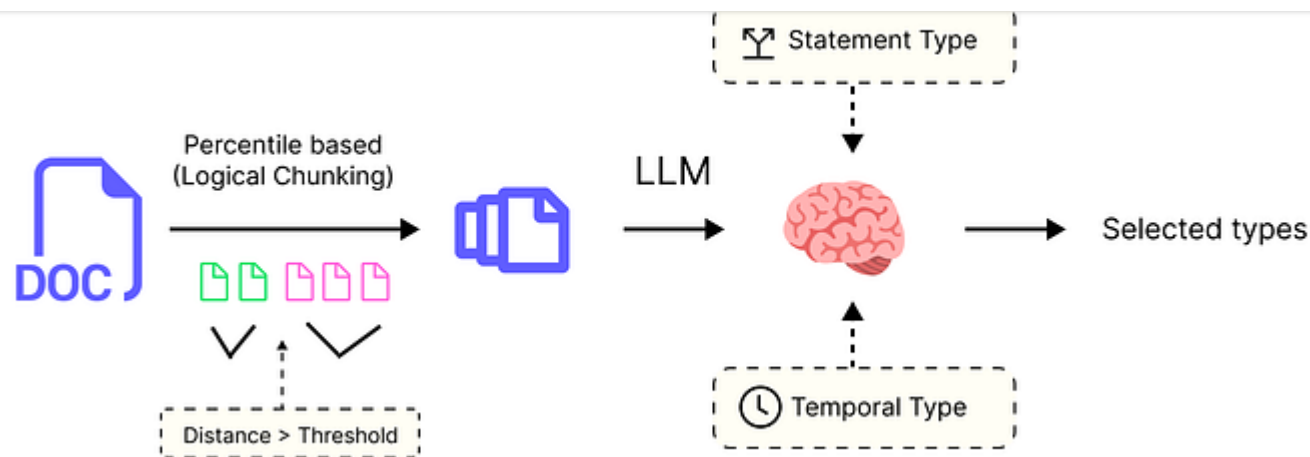
Now that we have our data neatly organized into small, meaningful chunks, we can begin using a LLM to read these chunks and pull out the core facts.



Atomic Facts Extraction (Created by [Fareed Khan](#))

Why Extract Statements First?

We need to break down the text into the smallest possible "atomic" facts. For example, instead of a single, complex sentence, we want individual claims that can stand on their own.

Atomic Facts Extraction Flow (Created by [Fareed Khan](#))

This process makes the information much easier for our AI system to understand, query, and reason with later on.

To ensure our LLM gives us a clean, predictable output, we need to give it a strict set of instructions. The best way to do this in Python is with `Pydantic` models. These models act as a "schema" or a "template" that the LLM must follow.

First, let's define the allowed categories for our labels using `Enums`. We are using `(str, Enum)`.

Copy

```
# Enum for temporal labels describing time sensitivity
class TemporalType(str, Enum):
    ATEMPORAL = "ATEMPORAL" # Facts that are always true (e.g., "Earth is a planet")
    STATIC = "STATIC" # Facts about a single point in time (e.g., "Product launched in 2020")
    DYNAMIC = "DYNAMIC" # Facts describing an ongoing state (e.g., "Lisa is the manager")
```

Each category captures a different kind of temporal reference:

- **Atemporal:** Statements that are universally true and invariant over time (e.g., "Water boils at 100 degrees Celsius.").
- **Static:** Statements that became true at a specific point in time and remain unchanged thereafter (e.g., "John was hired as manager on June 1, 2020.").
- **Dynamic:** Statements that may change over time and require temporal context to interpret accurately (e.g., "John is the manager of the team.").

Copy

```
# Enum for statement labels classifying statement nature
class StatementType(str, Enum):
    FACT = "FACT" # An objective, verifiable claim
    OPINION = "OPINION" # A subjective belief or judgment
    PREDICTION = "PREDICTION" # A statement about a future event
```

- **Fact:** A true statement at the time it's said, but might change later (e.g., "The company made \$5 million last quarter.").
- **Opinion:** A personal belief or feeling, true only when said (e.g., "I think this product will do well.").
- **Prediction:** A guess about the future, true from now until the predicted time ends (e.g., "Sales will grow next year.").

By defining these fixed categories, we ensure consistency in how our agent classifies the information it extracts.

Now, let's create the Pydantic models that will use these enums to define the structure for our output.

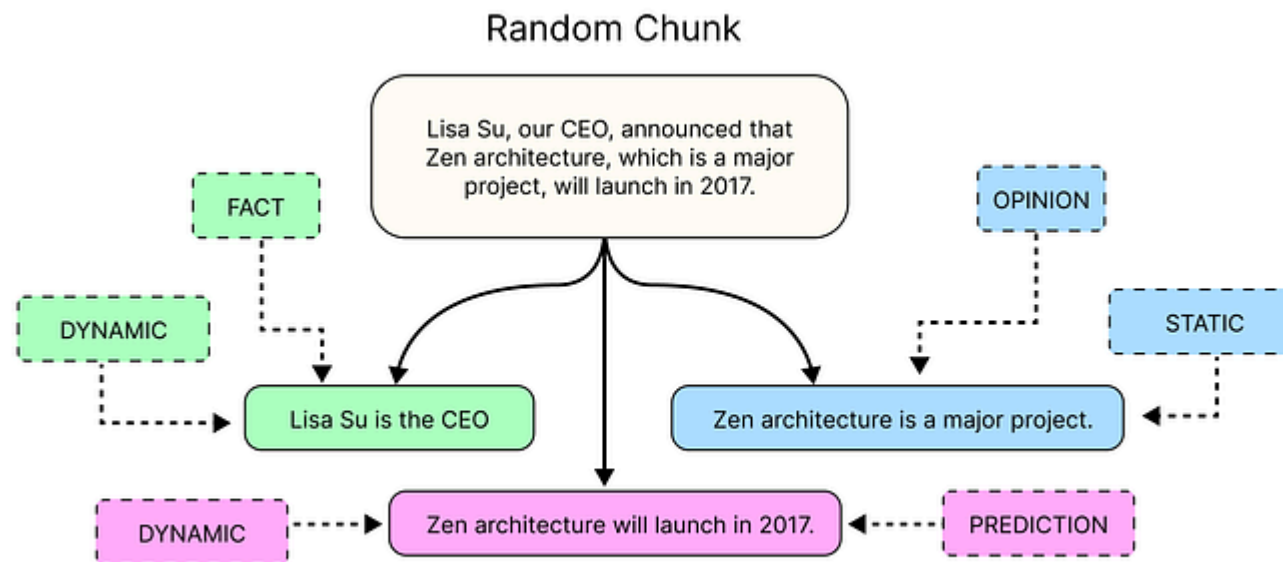
Copy

```
from pydantic import BaseModel, field_validator

# This model defines the structure for a single extracted statement
class RawStatement(BaseModel):
    statement: str
    statement_type: StatementType
    temporal_type: TemporalType

# This model is a container for the list of statements from one chunk
```

These models are our contract with the LLM. We are telling it, "When you're done processing a chunk, your answer must be a JSON object that contains a list called 'statements', and each item in that list must have a `statement`, a `statement_type`, and a `temporal_type`".



Atomic Facts Extraction (Created by [Fareed Khan](#))

or a STATIC and DYNAMIC statement.

Copy

```
# These definitions provide the necessary context for the LLM to understand the
LABEL_DEFINITIONS: dict[str, dict[str, dict[str, str]]] = {
    "episode_labelling": {
        "FACT": dict(definition="Statements that are objective and can be inde
        "OPINION": dict(definition="Statements that contain personal opinions,
        "PREDICTION": dict(definition="Uncertain statements about the future or
    },
    "temporal_labelling": {
        "STATIC": dict(definition="Often past tense, think -ed verbs, describin
        "DYNAMIC": dict(definition="Often present tense, think -ing verbs, desc
        "ATEMPORAL": dict(definition="Statements that will always hold true rec
    },
}
```

These definitions are just descriptions of each label. They provide better support to the LLM by explaining why it should assign a particular label to the extracted information. Now, we need to create the prompt template using these definitions.

Copy


```
for section_key, section_dict in LABEL_DEFINITIONS.items():
    # Add a section header with underscores replaced by spaces and uppercased
    definitions_text += f"==== {section_key.replace('_', ' ').upper()} DEFINITION\n\n"

    # Add each category and its definition under the section
    for category, details in section_dict.items():
        definitions_text += f"-- {category}: {details.get('definition', '')}\n"
```

This `definitions_text` string will be a key part of our prompt, giving the LLM the "textbook" definitions it needs to perform its task accurately.

Now, we will construct the main prompt template. This template brings everything together: the inputs, the task instructions, the label definitions, and a crucial example to show the LLM exactly what a good output looks like.

Notice how we use `{{` and `}}` to "escape" the curly braces in our JSON example. This tells LangChain that those are part of the text, not variables to be filled.

Copy

```
from langchain_core.prompts import ChatPromptTemplate

# Define the prompt template for statement extraction and labeling
```

Inputs:

- main_entity: {main_entity}
- document_chunk: {document_chunk}

Tasks:

1. Extract clear, single-subject statements.
2. Label each as FACT, OPINION, or PREDICTION.
3. Label each temporally as STATIC, DYNAMIC, or ATEMPORAL.
4. Resolve references to main_entity and include dates/quantities.

Return ONLY a JSON object with the statements and labels.

"""

```
# Create a ChatPromptTemplate from the string template
```

```
prompt = ChatPromptTemplate.from_template(statement_extraction_prompt_template)
```

Finally, we connect everything. We will create a LangChain "chain" that links our prompt to the LLM and tells the LLM to structure its output according to our `RawStatementList` model.

We will use the `deepseek-ai/DeepSeek-V3` model via Nebius for this task, as it's powerful and good at following complex instructions.

Copy

```
# Initialize our LLM
llm = ChatNebius(model="deepseek-ai/DeepSeek-V3")

# Create the chain: prompt -> LLM -> structured output parser
statement_extraction_chain = prompt | llm.with_structured_output(RawStatementL
```

Let's test our chain on a single chunk from our dataset to see it in action.

Copy

```
# Select the sample chunk we inspected earlier for testing extraction
sample_chunk_for_extraction = chunked_documents_lc[10]

print("--- Running statement extraction on a sample chunk ---")
print(f"Chunk Content:\n{sample_chunk_for_extraction.page_content}")
print("\nInvoking LLM for extraction...")

# Call the extraction chain with necessary inputs
extracted_statements_list = statement_extraction_chain.invoke({
    "main_entity": sample_chunk_for_extraction.metadata["company"],
    "publication_date": sample_chunk_for_extraction.metadata["date"].isoformat(),
    "document_chunk": sample_chunk_for_extraction.page_content,
    "definitions": definitions_text
})

print("\n--- Extraction Result ---")
```

Running the above code we got the following structured output for our chunks.

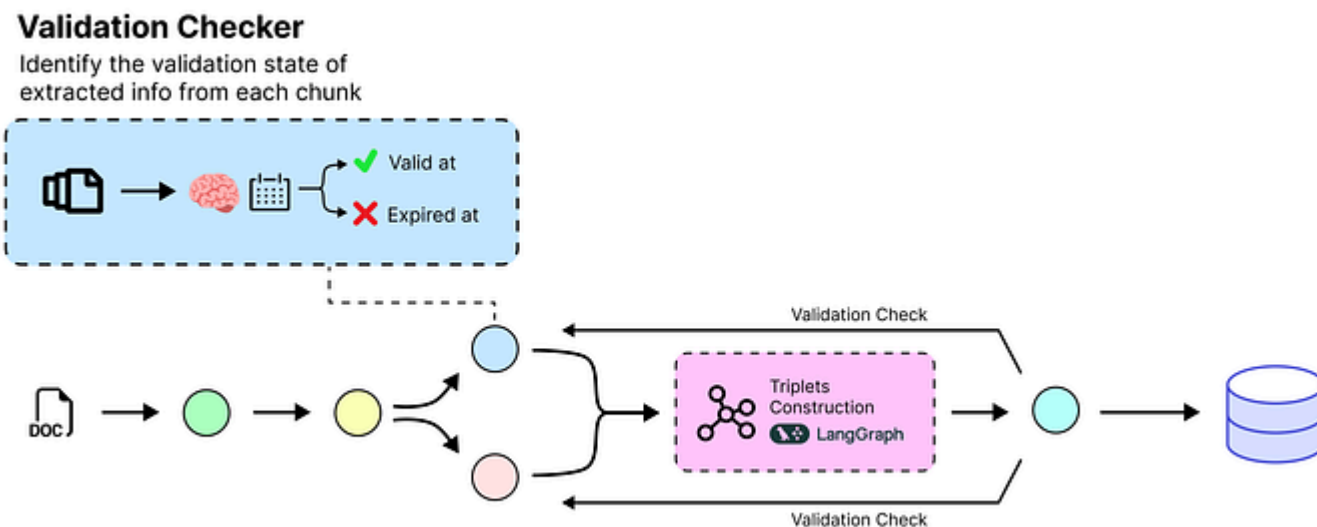
Copy

```
#### OUTPUT ####
{
  "statements": [
    {
      "statement": "AMD has been very focused on the server launch for the first",
      "statement_type": "FACT",
      "temporal_type": "DYNAMIC"
    },
    {
      "statement": "AMD's Desktop product should launch before the server launch",
      "statement_type": "PREDICTION",
      "temporal_type": "STATIC"
    },
    {
      "statement": "AMD believes true volume availability will be in the first",
      "statement_type": "OPINION",
      "temporal_type": "STATIC"
    },
    {
      "statement": "AMD may ship some limited volume towards the end of the fourth",
      "statement_type": "PREDICTION",
      "temporal_type": "STATIC"
    }
  ]
}
```

So far, we have completed our atomic facts step, where we accurately extract temporal and statement-based facts that can later be updated if needed.

Pinpointing Time with a Validation Check Agent

We have successfully extracted the **what** from our text the atomic statements. Now, we need to extract the **when**. Each statement needs a precise timestamp to tell us when it was valid.



Was a product launched last week or last year? or Was a CEO in her role in 2016 or 2018?

This is the most important step in making our knowledge base truly "temporal".

Why We Need a Dedicated Date Agent?

Extracting dates from natural language is tricky. A statement might say **next quarter, three months ago or in 2017**. Humans understand this, but a computer needs a concrete date, like `2017-01-01` .

Our goal is to create a specialized agent that can read a statement and, using the original document's publication date as a reference, figure out two key timestamps:

- `valid_at` : The date the fact became true.
- `invalid_at` : The date the fact stopped being true (if it's no longer valid).

Just like before, we'll start by defining Pydantic models to ensure our LLM gives us a clean, structured output for the dates.

convert them into proper Python datetime objects.

Copy

```
from datetime import datetime, timezone
from dateutil.parser import parse
import re

def parse_date_str(value: str | datetime | None) -> datetime | None:
    """
    Parse a string or datetime into a timezone-aware datetime object (UTC).
    Returns None if parsing fails or input is None.
    """
    if not value:
        return None

    # If already a datetime, ensure it has timezone info (UTC if missing)
    if isinstance(value, datetime):
        return value if value.tzinfo else value.replace(tzinfo=timezone.utc)

    try:
        # Handle year-only strings like "2023"
        if re.fullmatch(r"\d{4}", value.strip()):
            year = int(value.strip())
            return datetime(year, 1, 1, tzinfo=timezone.utc)

        # Parse more complex date strings with dateutil
        dt: datetime = parse(value)
```

```
if dt.tzinfo is None:
    dt = dt.replace(tzinfo=timezone.utc)

    return dt
except Exception:
    return None
```

Now, we'll define two Pydantic models. `RawTemporalRange` will capture the raw text output from the LLM, and `TemporalValidityRange` will use our helper function to automatically parse that text into clean `datetime` objects.

Copy

```
from pydantic import BaseModel, Field, field_validator
from datetime import datetime

# Model for raw temporal range with date strings as ISO 8601
class RawTemporalRange(BaseModel):
    valid_at: str | None = Field(None, description="The start date/time in ISO")
    invalid_at: str | None = Field(None, description="The end date/time in ISO")

# Model for validated temporal range with datetime objects
class TemporalValidityRange(BaseModel):
    valid_at: datetime | None = None
    invalid_at: datetime | None = None

    # Validator to parse date strings into datetime objects before assignment
```



```
def _parse_date_string(text, value: str | datetime | None) -> datetime | None:
    return parse_date_str(value)
```

This two-step model approach is a great practice. It separates the raw LLM output from our clean, validated application data, making the pipeline more robust.

Next, we create a new prompt specifically for this date extraction task. We will give the LLM one of our extracted statements and ask it to determine the validity dates based on the context.

Copy

```
# Prompt guiding the LLM to extract temporal validity ranges from statements
date_extraction_prompt_template = """
You are a temporal information extraction specialist.
```

INPUTS:

- statement: "{statement}"
- statement_type: "{statement_type}"
- temporal_type: "{temporal_type}"
- publication_date: "{publication_date}"
- quarter: "{quarter}"

TASK:

- Analyze the statement and determine the temporal validity range (valid_at, in
- Use the publication date as the reference point for relative expressions (e.g

GUIDANCE:

- For STATIC statements, `valid\_at` is the date the event occurred, and `invalid\_at` is the date the event ended.
- For DYNAMIC statements, `valid\_at` is when the state began, and `invalid\_at` is when the state ended.
- Return dates in ISO 8601 format (e.g., YYYY-MM-DDTHH:MM:SSZ).

****Output format****

Return ONLY a valid JSON object matching the schema for `RawTemporalRange`.

"""

Create a LangChain prompt template from the string

date_extraction_prompt = ChatPromptTemplate.from_template(date_extraction_prompt)

This prompt is highly specialized. It tells the LLM to act like a "temporal specialist" and gives it clear rules for how to handle different types of statements.

Now we build the LangChain chain for this step and test it on one of the statements we extracted earlier.

Copy

Reuse the existing LLM instance.

Create a chain by connecting the date extraction prompt

with the LLM configured to output structured RawTemporalRange objects.

date_extraction_chain = date_extraction_prompt | llm.with_structured_output(RawTemporalRange)

AMD has been very focused on the server launch for the first half of 2017.

This is a `DYNAMIC` fact, so we expect a `valid_at` date, but the end date is not specified.

Copy

```
# Take the first extracted statement for date extraction testing
sample_statement = extracted_statements_list.statements[0]
chunk_metadata = sample_chunk_for_extraction.metadata

print(f"--- Running date extraction for statement ---")
print(f'Statement: "{sample_statement.statement}"')
print(f'Reference Publication Date: {chunk_metadata["date"].isoformat()}')

# Invoke the date extraction chain with relevant inputs
raw_temporal_range = date_extraction_chain.invoke({
    "statement": sample_statement.statement,
    "statement_type": sample_statement.statement_type.value,
    "temporal_type": sample_statement.temporal_type.value,
    "publication_date": chunk_metadata["date"].isoformat(),
    "quarter": chunk_metadata["quarter"]
})

# Parse and validate raw LLM output into a structured TemporalValidityRange model
final_temporal_range = TemporalValidityRange.model_validate(raw_temporal_range)
```

```
print(f"Valid At: {final_temporal_range.valid_at} ",  
      print(f"Invalid At: {final_temporal_range.invalid_at}")
```

Let's see how the extracted timeframe fact looks like:

Copy

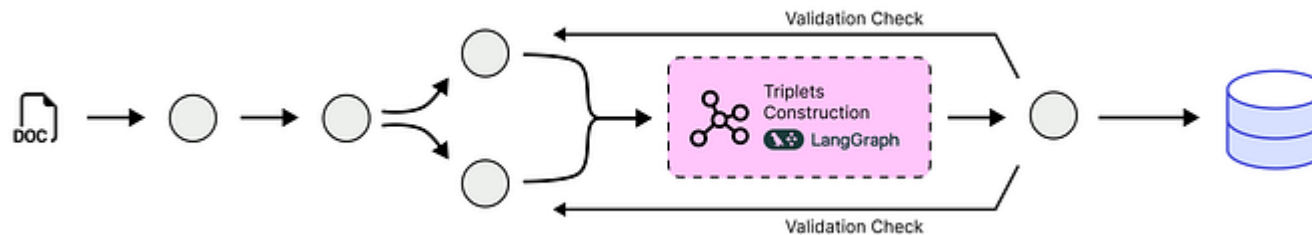
```
##### OUTPUT #####  
--- Running date extraction for statement ---  
Statement: "AMD has been very focused on the server launch for the first half of 2017"  
Reference Publication Date: 2016-07-21  
  
--- Parsed & Validated Result ---  
Valid At: 2017-01-01 00:00:00+00:00  
Invalid At: 2017-06-30 00:00:00+00:00
```

The LLM correctly interpreted "first half of 2017" and converted it into a precise date range. It understood that this `DYNAMIC` statement had a clear start and end.

We have now successfully added a time dimension to our facts. The next step is to break down the text of the statements even further into the fundamental structure of a knowledge graph, **Triplets**.

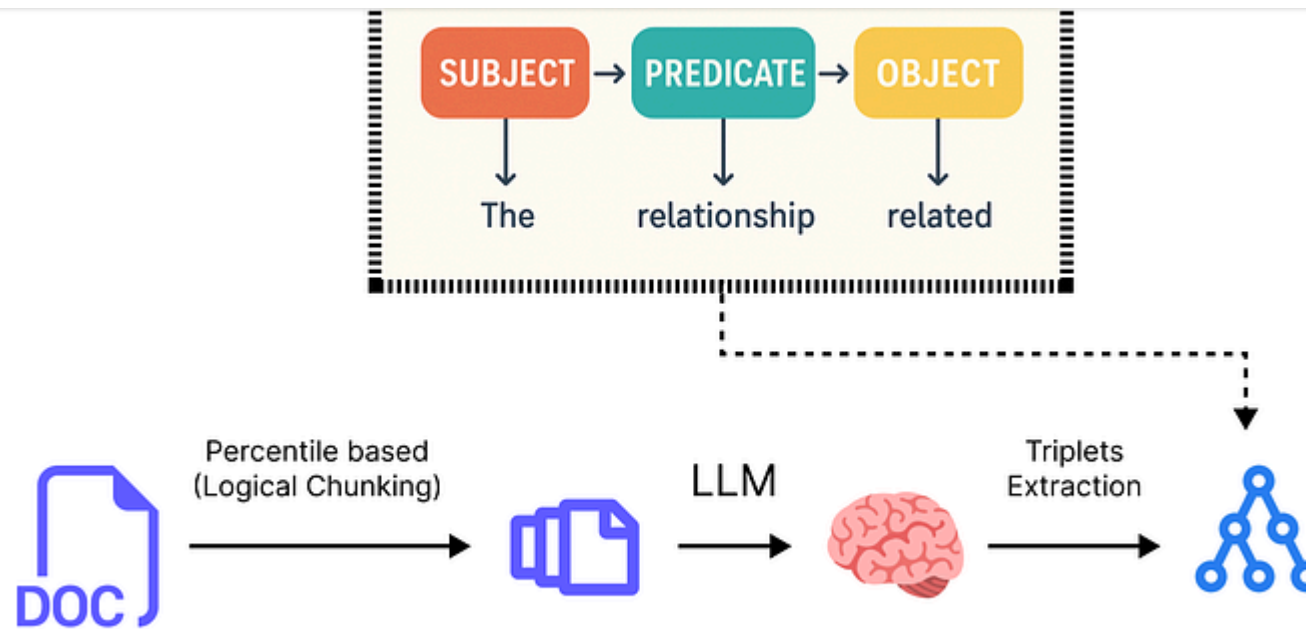
Structuring Facts into Triplets

We have successfully extracted what the facts are (the statements) and when they were true (the dates).



Triplets Extraction (Created by [Fareed Khan](#))

Now, we need to convert these natural language sentences into a format that an AI agent can easily understand and connect.

Triplet Extraction Flow (Created by [Fareed Khan](#))

A triplet breaks down a fact into three core components:

1. **Subject:** The main entity the fact is about.
2. **Predicate:** The relationship or action.
3. **Object:** The entity or concept the subject is related to.

As before, we will start by defining the `Pydantic` models that will structure the output from our LLM. This extraction is our most complex yet, as the LLM needs to identify both the entities (the nouns) and the relationships (the triplets) at the same time.

First, let's define the fixed list of `Predicate` relationships we want our agent to use. This ensures consistency across our entire knowledge graph.

Copy

```
from enum import Enum # Import the Enum base class to create enumerated constants

# Enum representing a fixed set of relationship predicates for graph consistency
class Predicate(str, Enum):
    # Each member of this Enum represents a specific type of relationship between entities
    IS_A = "IS_A" # Represents an "is a" relationship (e.g., a Dog is a Mammal)
    HAS_A = "HAS_A" # Represents possession or composition (e.g., a Car has an Engine)
    LOCATED_IN = "LOCATED_IN" # Represents location relationship (e.g., A Store is located in a City)
    HOLDS_ROLE = "HOLDS_ROLE" # Represents role or position held (e.g., A Person holds a Job)
    PRODUCES = "PRODUCES" # Represents production or creation relationship (e.g., A Factory produces Cars)
    SELLS = "SELLS" # Represents selling relationship between entities (e.g., A Store sells Products)
    LAUNCHED = "LAUNCHED" # Represents launch events (e.g., A Product is launched by a Company)
    DEVELOPED = "DEVELOPED" # Represents development relationship (e.g., A Software is developed by a Team)
    ADOPTED_BY = "ADOPTED_BY" # Represents adoption relationship (e.g., A Policy is adopted by a Government)
```

```

SUPPLIES = "SUPPLIES" # Represents supplier relationship (e.g., Supp
HAS_REVENUE = "HAS_REVENUE" # Represents revenue relationship for entities
INCREASED = "INCREASED" # Represents an increase event or metric change
DECREASED = "DECREASED" # Represents a decrease event or metric change
RESULTED_IN = "RESULTED_IN" # Represents causal relationship (e.g., Event
TARGETS = "TARGETS" # Represents target or goal relationship
PART_OF = "PART_OF" # Represents part-whole relationship (e.g., Wh
DISCONTINUED = "DISCONTINUED" # Represents discontinued status or event
SECURED = "SECURED" # Represents secured or obtained relationship

```

Now, we define the models for the raw entities and triplets, and a container model, `RawExtraction`, to hold the entire output.

Copy

```

from pydantic import BaseModel, Field
from typing import List, Optional

# Model representing an entity extracted by the LLM
class RawEntity(BaseModel):
    entity_idx: int = Field(description="A temporary, 0-indexed ID for this entity")
    name: str = Field(description="The name of the entity, e.g., 'AMD' or 'Lisa")
    type: str = Field("Unknown", description="The type of entity, e.g., 'Organ")
    description: str = Field("", description="A brief description of the entity")

# Model representing a single subject-predicate-object triplet
class RawTriplet(BaseModel):

```



```

predicate: Predicate
    object_name: str
    object_id: int = Field(description="The entity_idx of the object.")
    value: Optional[str] = Field(None, description="An optional value, e.g., '1'")

# Container for all entities and triplets extracted from a single statement
class RawExtraction(BaseModel):
    entities: List[RawEntity]
    triplets: List[RawTriplet]

```

This structure is very clever. It asks the LLM to first list all the entities it finds, giving each a temporary number (`entity_idx`).

Then, it asks the LLM to build triplets using those numbers, creating a clear link between the relationships and the entities involved.

Next, we create the prompt and the definitions that will guide the LLM. This prompt is highly specific, instructing the model to focus only on the relationships and ignore any time-based information, which we have already extracted.

Copy

```

# These definitions guide the LLM in choosing the correct predicate.
PREDICATE_DEFINITIONS = {
    "IS_A": "Denotes a class-or-type relationship (e.g., 'Model Y IS_A electric car')."
}

```

```
    HOLDS_ROLE : connects a person to a formal office or title. ,  
    }  
  
    # Format the predicate instructions into a string for the prompt.  
    predicate_instructions_text = "\n".join(f"- {pred}: {desc}" for pred, desc in F
```

Now for the main prompt template. Again, we are careful to escape the JSON example with `{{` and `}}` so LangChain can parse it correctly.

Copy

```
# Prompt for extracting entities and subject-predicate-object triplets from a s  
triplet_extraction_prompt_template = """  
You are an information-extraction assistant.  
  
Task: From the statement, identify all entities (people, organizations, product  
  
Statement: "{statement}"  
  
Predicate list:  
{predicate_instructions}  
  
Guidelines:  
- List entities with unique `entity_idx`.  
- List triplets linking subjects and objects by `entity_idx`.  
- Exclude temporal expressions from entities and triplets.
```

```

output: {
  "entities": [
    {"entity_idx": 0, "name": "Google", "type": "Organization", "description": "Google is a multinational technology company."},
    {"entity_idx": 1, "name": "Revenue", "type": "Financial Metric", "description": "Revenue is a financial metric representing the total amount of money generated by a company."},
  ],
  "triplets": [
    {"subject_name": "Google", "subject_id": 0, "predicate": "INCREASED", "object": "Revenue"}
  ]
}

```

Return ONLY a valid JSON object matching `RawExtraction`.

"""

Initializing the prompt template

triplet_extraction_prompt = ChatPromptTemplate.from_template(triplet_extraction_prompt)

Finally, we create our third chain and test it on one of our statements.

Copy

```

# Create the chain for triplet and entity extraction.
triplet_extraction_chain = triplet_extraction_prompt | llm.with_structured_output(RawExtraction)

```

Let's use the same statement we've been working with.

```

sample_statement_for_triplets = extracted_statements_list.statements[0]

```

```

print(f"--- Running triplet extraction for statement ---")

```

```

print(f'Statement: "{sample_statement_for_triplets.statement}")

```

```
raw_extraction_result = triplet_extraction_chain.invoke({
    "statement": sample_statement_for_triplets.statement,
    "predicate_instructions": predicate_instructions_text
})

print("\n--- Triplet Extraction Result ---")
print(raw_extraction_result.model_dump_json(indent=2))
```

This is what our chain output is.

Copy

```
--- Running triplet extraction for statement ---
Statement: "AMD has been very focused on the server launch for the first half of 2024"

--- Triplet Extraction Result ---
{
  "entities": [
    {
      "entity_idx": 0,
      "name": "AMD",
      "type": "Organization",
      "description": ""
    },
    {
      "entity_idx": 1,
      "name": "server launch",
      "type": "Event",
      "description": ""
    }
  ]
}
```

```
,  
  "triplets": [  
    {  
      "subject_name": "AMD",  
      "subject_id": 0,  
      "predicate": "TARGETS",  
      "object_name": "server launch",  
      "object_id": 1,  
      "value": null  
    }  
  ]  
}
```

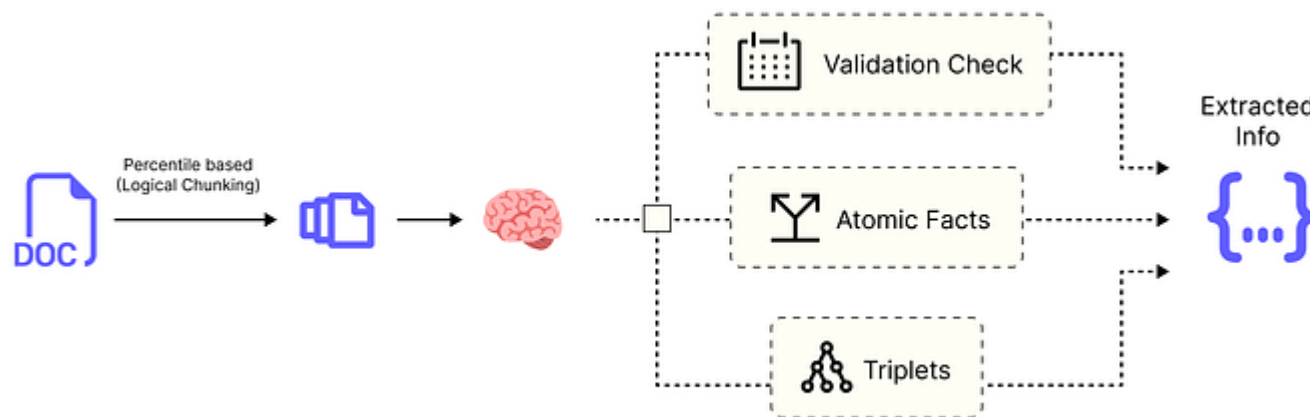
The LLM correctly identified "AMD" and "server launch" as the key entities and linked them with the `TARGETS` predicate, perfectly capturing the meaning of the original sentence.

We have now completed all the individual extraction steps. We have a system that can take a piece of text and extract statements, dates, entities, and triplets. The next step is to combine all this information into a single, unified object that represents a complete **"Temporal Event"**.

Assembling the Temporal Event

1. **Statements:** The atomic facts.
2. **Dates:** The "when" for each fact.
3. **Entities & Triplets:** The "who" and "what" in a structured format.

The final step in our extraction process is to bring all these pieces together. We will create a master data model called a TemporalEvent that consolidates all the information about a single statement into one clean, unified object.



Temporal Event (Created by [Fareed Khan](#))

temporal range, and links to all the triplets that were derived from it.

Let's also define the final, persistent versions of our `Triplet` and `Entity` models. These are almost identical to the `Raw` versions, but they will use a `uuid` (a unique universal identifier) for their primary IDs. This is a best practice that prepares them for storage in any database.

Copy

```
import uuid
from pydantic import BaseModel, Field

# Final persistent model for an entity in your knowledge graph
class Entity(BaseModel):
    id: uuid.UUID = Field(default_factory=uuid.uuid4, description="Unique UUID")
    name: str = Field(..., description="The name of the entity")
    type: str = Field(..., description="Entity type, e.g., 'Organization', 'Person')
    description: str = Field("", description="Brief description of the entity")
    resolved_id: uuid.UUID | None = Field(None, description="UUID of resolved entity")

# Final persistent model for a triplet relationship
class Triplet(BaseModel):
    id: uuid.UUID = Field(default_factory=uuid.uuid4, description="Unique UUID")
    subject_name: str = Field(..., description="Name of the subject entity")
    subject_id: uuid.UUID = Field(..., description="UUID of the subject entity")
    predicate: Predicate = Field(..., description="Relationship predicate")
```

value: `str | None` = Field(`None`, description="Optional value associated with

Now for the main `TemporalEvent` model. It includes all the information we've extracted so far, plus some extra fields for embeddings and invalidation that we will use in later steps.

Copy

```
class TemporalEvent(BaseModel):
    """
    The central model that consolidates all extracted information.
    """
    id: uuid.UUID = Field(default_factory=uuid.uuid4)
    chunk_id: uuid.UUID # To link back to the original text chunk
    statement: str
    embedding: list[float] = [] # For similarity checks later

    # Information from our previous extraction steps
    statement_type: StatementType
    temporal_type: TemporalType
    valid_at: datetime | None = None
    invalid_at: datetime | None = None

    # A list of the IDs of the triplets associated with this event
    triplets: list[uuid.UUID]

    # Extra metadata for tracking changes over time
```


Invalidated_by: uuid:UUID | NONE - NONE

Let's manually assemble a single `TemporalEvent` to see how all the pieces fit together. We'll use the results from the previous steps for our sample statement:

AMD has been very focused on the server launch for the first half of 2017.

First, we will convert our `RawEntity` and `RawTriplet` objects into their final, persistent `Entity` and `Triplet` forms, complete with unique UUIDs.

Copy

```
# Assume these are already defined from previous steps:
# sample_statement, final_temporal_range, raw_extraction_result

print("--- Assembling the final TemporalEvent ---")

# 1. Convert raw entities to persistent Entity objects with UUIDs
idx_to_entity_map: dict[int, Entity] = {}
final_entities: list[Entity] = []

for raw_entity in raw_extraction_result.entities:
    entity = Entity(
        name=raw_entity.name,
```

```
,
idx_to_entity_map[raw_entity.entity_idx] = entity
final_entities.append(entity)

print(f"Created {len(final_entities)} persistent Entity objects.")

# 2. Convert raw triplets to persistent Triplet objects, linking entities via l
final_triplets: list[Triplet] = []

for raw_triplet in raw_extraction_result.triplets:
    subject_entity = idx_to_entity_map[raw_triplet.subject_id]
    object_entity = idx_to_entity_map[raw_triplet.object_id]

    triplet = Triplet(
        subject_name=subject_entity.name,
        subject_id=subject_entity.id,
        predicate=raw_triplet.predicate,
        object_name=object_entity.name,
        object_id=object_entity.id,
        value=raw_triplet.value
    )
    final_triplets.append(triplet)

print(f"Created {len(final_triplets)} persistent Triplet objects.")
```

With our final Entity and Triplet objects ready, we can now construct the master TemporalEvent record.

```
# 5. Create the final TemporalEvent object
# We'll generate a dummy chunk_id for this example.
temporal_event = TemporalEvent(
    chunk_id=uuid.uuid4(), # Placeholder ID
    statement=sample_statement.statement,
    statement_type=sample_statement.statement_type,
    temporal_type=sample_statement.temporal_type,
    valid_at=final_temporal_range.valid_at,
    invalid_at=final_temporal_range.invalid_at,
    triplets=[t.id for t in final_triplets]
)

print("\n--- Final Assembled TemporalEvent ---")
print(temporal_event.model_dump_json(indent=2))

print("\n--- Associated Entities ---")
for entity in final_entities:
    print(entity.model_dump_json(indent=2))

print("\n--- Associated Triplets ---")
for triplet in final_triplets:
    print(triplet.model_dump_json(indent=2))
```

Let's take a look at the pipeline result.

Copy

```
"id": "d6640945-8404-476f-bcf2-1ad5889f5321",
"chunk_id": "3543e983-8ddf-4e7e-9833-9476dc747f6d",
"statement": "AMD has been very focused on the server launch for the first ha
"embedding": [],
"statement_type": "FACT",
"temporal_type": "DYNAMIC",
"valid_at": "2017-01-01T00:00:00+00:00",
"invalid_at": "2017-06-30T00:00:00+00:00",
"triplets": [
  "af3a84b0-4430-424c-858f-650ad3d211e0"
],
"created_at": "2024-08-10T19:17:40.509077",
"expired_at": null,
"invalidated_by": null
}

--- Associated Entities ---
{
  "id": "a7e56f1c-caba-4a07-b582-7feb9cf1a48c",
  "name": "AMD",
  "type": "Organization",
  "description": "",
  "resolved_id": null
}
{
  "id": "582c4edd-7310-4570-bc4f-281db179c673",
  "name": "server launch",
  "type": "Event",
  "description": "",
```

```
--- Associated Triplets ---
{
  "id": "af3a84b0-4430-424c-858f-650ad3d211e0",
  "subject_name": "AMD",
  "subject_id": "a7e56f1c-caba-4a07-b582-7feb9cf1a48c",
  "predicate": "TARGETS",
  "object_name": "server launch",
  "object_id": "582c4edd-7310-4570-bc4f-281db179c673",
  "value": null
}
```

We have successfully traced a single piece of information from a raw paragraph all the way to a fully structured, time-stamped `TemporalEvent` .

But doing this manually for every statement would be impossible. The next step is to automate this entire assembly line using `LangGraph` to process all our chunks at once.

Automating the Pipeline with LangGraph

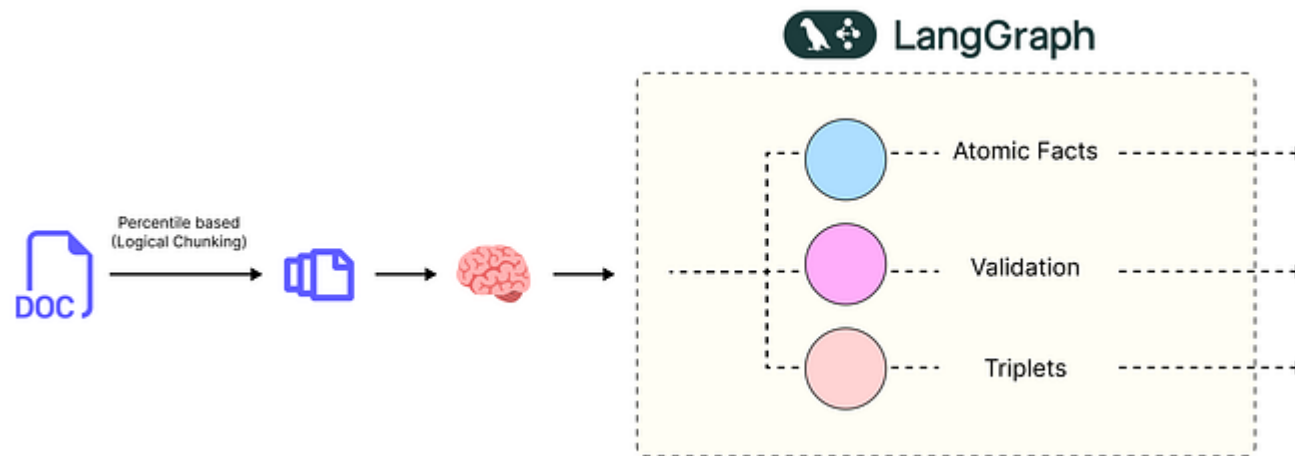
So far, we have built three powerful, specialized "agents" (or chains): one to extract statements, one for dates, and one for triplets. We've also seen how to manually combine their outputs into a final `TemporalEvent` .

comes in.

What is LangGraph and Why Use It?

LangGraph is a library from LangChain for building complex, stateful AI applications. Instead of a simple `prompt -> LLM chain`, we can build a *graph* where each step is a "node". The information, or "state" flows from one node to the next.

This is perfect for our use case. We can create a graph where:



2. The second node extracts dates for all those statements.
3. The third node extracts triplets.
4. And so on...

This creates a robust, repeatable, and easy-to-debug pipeline for processing our data.

The first step in building a LangGraph is to define its "state." The state is the memory of the graph, it's the data that gets passed between each node. We'll define a state that can hold our list of chunks and all the new objects we create along the way.

Copy

```
from typing import List, TypedDict
from langchain_core.documents import Document

class GraphState(TypedDict):
    """
    TypedDict representing the overall state of the knowledge graph ingestion.

    Attributes:
        chunks: List of Document chunks being processed.
```

```
triplets: List of Triplet objects representing relationships.

"""
chunks: List[Document]
temporal_events: List[TemporalEvent]
entities: List[Entity]
triplets: List[Triplet]
```

This simple dictionary structure will act as the "conveyor belt" for our factory assembly line, carrying our data from one station to the next.

Now, we'll combine all our previous logic into a single, powerful function. This function will be the main "node" in our graph. It takes the list of chunks from the state and orchestrates the entire extraction process in three parallel steps.

A key optimization here is using the `.batch()` method. Instead of processing statements one by one in a slow loop, `.batch()` sends all of them to the LLM at once, which is much faster and more efficient.

Copy

```
def extract_events_from_chunks(state: GraphState) -> GraphState:
    chunks = state["chunks"]

    # Extract raw statements from each chunk
```



```
        "publication_date": c.metadata["date"].isoformat(),
        "document_chunk": c.page_content,
        "definitions": definitions_text
    } for c in chunks])

# Flatten statements, attach metadata and unique chunk IDs
stmts = [{"raw": s, "meta": chunks[i].metadata, "cid": uuid.uuid4()}
         for i, rs in enumerate(raw_stmts) for s in rs.statements]

# Prepare inputs and batch extract temporal data
dates = date_extraction_chain.batch([
    {"statement": s["raw"].statement,
     "statement_type": s["raw"].statement_type.value,
     "temporal_type": s["raw"].temporal_type.value,
     "publication_date": s["meta"]["date"].isoformat(),
     "quarter": s["meta"]["quarter"]}
    for s in stmts])

# Prepare inputs and batch extract triplets
trips = triplet_extraction_chain.batch([
    {"statement": s["raw"].statement,
     "predicate_instructions": predicate_instructions_text}
    for s in stmts])

events, entities, triplets = [], [], []

for i, s in enumerate(stmts):
    # Validate temporal range data
    tr = TemporalValidityRange.model_validate(dates[i].model_dump())
```

```

# Map entities by index and collect them
idx_map = {e.entity_idx: Entity(e.name, e.type, e.description) for e in
entities.extend(idx_map.values())

# Build triplets only if subject and object entities exist
tpls = [Triplet(
    idx_map[t.subject_id].name, idx_map[t.subject_id].id, t.predicate,
    idx_map[t.object_id].name, idx_map[t.object_id].id, t.value)
    for t in ext.triplets if t.subject_id in idx_map and t.object_id in
triplets.extend(tpls)

# Create TemporalEvent with linked triplet IDs
events.append(TemporalEvent(
    chunk_id=s["cid"], statement=s["raw"].statement,
    statement_type=s["raw"].statement_type, temporal_type=s["raw"].temp
    valid_at=tr.valid_at, invalid_at=tr.invalid_at,
    triplets=[t.id for t in tpls]
))

return {"chunks": chunks, "temporal_events": events, "entities": entities,

```

This function is the complete, automated version of the manual steps we performed earlier. It encapsulates the entire extraction logic.

Now, we can define our workflow. For now, it will be a simple graph with just one step: `extract_events` . We will add more steps for cleaning the data later.

```
from langgraph.graph import StateGraph, END

# Define a new graph using our state
workflow = StateGraph(GraphState)

# Add our function as a node named "extract_events"
workflow.add_node("extract_events", extract_events_from_chunks)

# Define the starting point of the graph
workflow.set_entry_point("extract_events")

# Define the end point of the graph
workflow.add_edge("extract_events", END)

# Compile the graph into a runnable application
app = workflow.compile()
```

Let's run our new automated pipeline on all the chunks we created from our single sample document.

Copy

```
# The input is a dictionary matching our GraphState, providing the initial chunks
graph_input = {"chunks": chunked_documents_lc}

# Invoke the graph. This will run our entire extraction pipeline in one call.
final_state = app.invoke(graph_input)
```

```
##### OUTPUT #####
```

```
--- Entering Node: extract_events_from_chunks ---  
Processing 19 chunks...  
Extracted a total of 95 statements from all chunks.  
Completed batch extraction for dates and triplets.  
Assembled 95 TemporalEvents, 213 Entities, and 121 Triplets.  
  
--- Graph execution complete ---
```

With a single `.invoke()` call, our LangGraph app processed all 19 chunks from our document, ran hundreds of parallel LLM calls, and assembled all the results into a clean, final state.

Let's inspect the output to see the scale of what we've accomplished.

Copy

```
# Check the number of objects created in the final state  
num_events = len(final_state['temporal_events'])  
num_entities = len(final_state['entities'])  
num_triplets = len(final_state['triplets'])  
  
print(f"Total TemporalEvents created: {num_events}")  
print(f"Total Entities created: {num_entities}")  
print(f"Total Triplets created: {num_triplets}")
```

```
print(FinalState["temporal_events"][0].model_dump_json(indent=2))
```

This is what the output of our sample processed data looks like.

Copy

```
Total TemporalEvents created: 95
Total Entities created: 213
Total Triplets created: 121

--- Sample TemporalEvent from the final state ---
{
  "id": "f7428490-a92a-4f0a-90f0-073b7d4f170a",
  "chunk_id": "37830de8-d442-45ae-84e4-0ae31ed1689f",
  "statement": "Jaguar Bajwa is an Analyst at Arete Research.",
  "embedding": [],
  "statement_type": "FACT",
  "temporal_type": "STATIC",
  "valid_at": "2016-07-21T00:00:00+00:00",
  "invalid_at": null,
  "triplets": [
    "87b60b81-fc4c-4958-a001-63f8b2886ea0"
  ],
  "created_at": "2025-08-10T19:52:48.580874",
  "expired_at": null,
  "invalidated_by": null
}
```

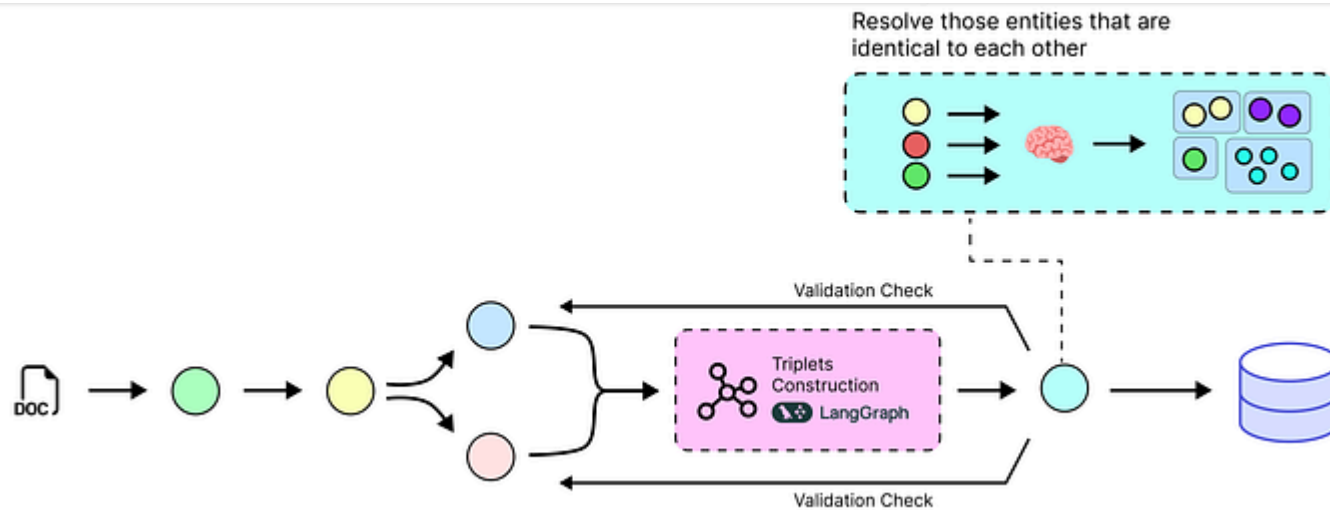
Devices" as two separate entities. The next step is to add a quality control station to our assembly line: **Entity Resolution**.

Cleaning Our Data with Entity Resolution

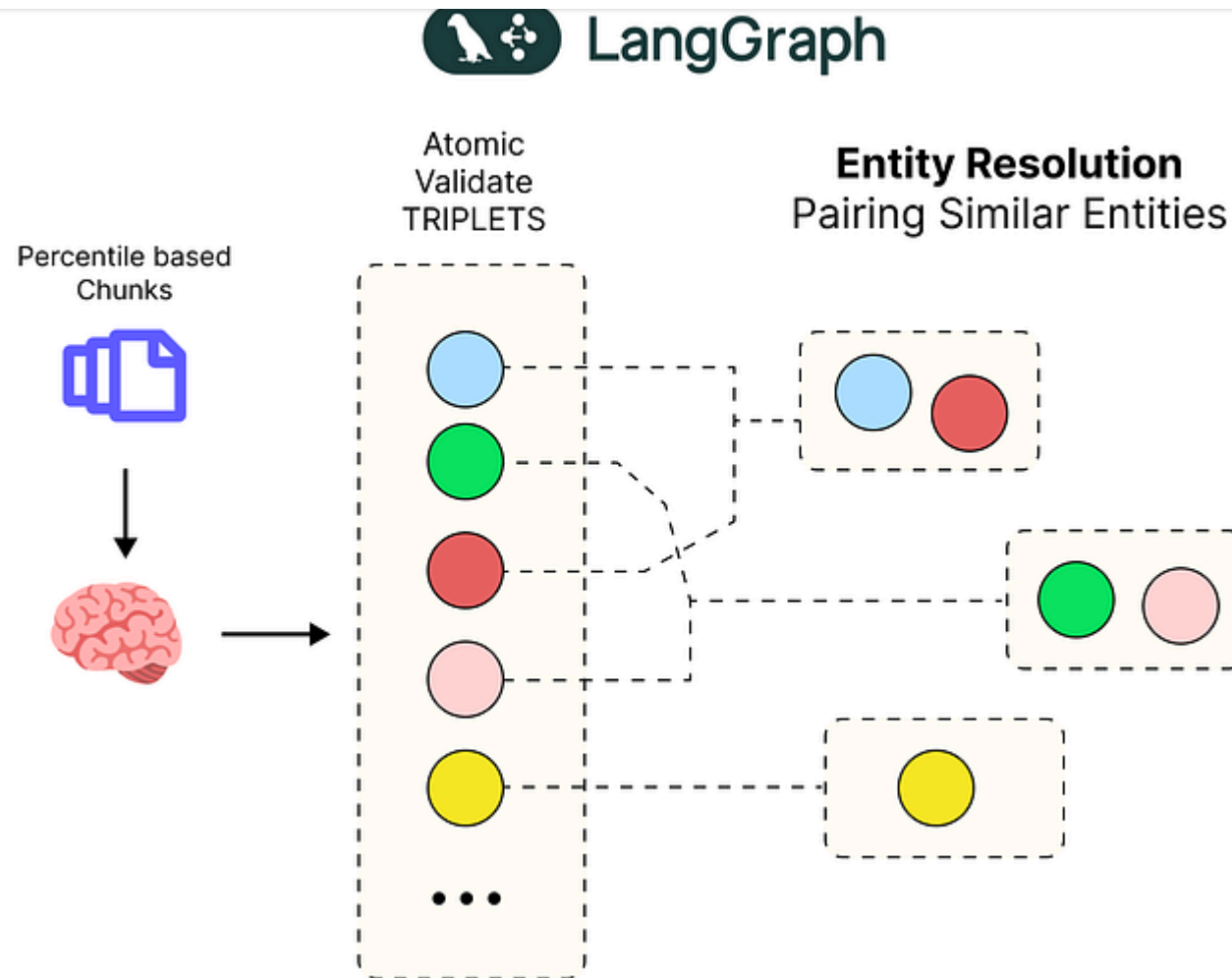
Our automated pipeline is now extracting a good amount of information. However, if you look closely at the entities, you might notice a problem.

An LLM might extract "AMD", "Advanced Micro Devices", and "Advanced Micro Devices, Inc" from different parts of the text.

To a human, these are obviously the same company, but to a computer, they are just different strings.

Entity Resolution (Created by [Fareed Khan](#))

This is a critical problem. If we don't fix it, our knowledge graph will be messy and unreliable. Queries about "AMD" would miss facts connected to "Advanced Micro Devices".



Entity Resolution (Created by [Fareed Khan](#))

This is known as **Entity Resolution**. The goal is to identify all the different names (or "mentions") that refer to the same real-world entity and merge them under a

To solve this, we will add a new quality control station to our LangGraph assembly line. This node will:

1. Cluster entities with similar names using fuzzy string matching.
2. Assign a single, canonical ID to all entities in a cluster.
3. Update our triplets to use these new, clean IDs.

To keep track of our canonical entities, we need a place to store them. For this tutorial, we'll create a simple, in-memory database using Python's built-in `sqlite3` library. In a real-world application, this would be a persistent, production-grade database.

Let's create our in-memory database and the table we'll need.

Copy

```
import sqlite3

def setup_in_memory_db():
    """
    Sets up an in-memory SQLite database and creates the 'entities' table.

    The 'entities' table schema:
```

```
- type: TEXT, type/category of the entity
- description: TEXT, description of the entity
- is_canonical: INTEGER, flag to indicate if entity is canonical (default 1)
```

Returns:

```
    sqlite3.Connection: A connection object to the in-memory database.
    """
```

```
# Establish connection to an in-memory SQLite database
conn = sqlite3.connect(":memory:")
```

```
# Create a cursor object to execute SQL commands
cursor = conn.cursor()
```

```
# Create the 'entities' table if it doesn't already exist
cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS entities (
        id TEXT PRIMARY KEY,
        name TEXT,
        type TEXT,
        description TEXT,
        is_canonical INTEGER DEFAULT 1
    )
    """)
```

```
# Commit changes to save the table schema
conn.commit()
```

```
# Return the connection object for further use
return conn
```

```
db_conn = Setup_In_Memory_Db()
```

This simple database will act as our central registry for all the clean, authoritative entities our agent identifies.

Now we'll create the function for our new LangGraph node. This function will contain the logic for finding and merging duplicate entities. For the fuzzy string matching, we'll use a handy library called `rapidfuzz`. You can simply install it using `pip install rapidfuzz`.

We have to write the `resolve_entities_in_state` function that will implement the clustering and merging logic.

Copy

```
import string
from rapidfuzz import fuzz
from collections import defaultdict

def resolve_entities_in_state(state: GraphState) -> GraphState:
    """
    A LangGraph node to perform entity resolution on the extracted entities.
    """
    print("\n--- Entering Node: resolve_entities_in_state ---")
```

```
cursor = db_conn.cursor()
cursor.execute("SELECT id, name FROM entities WHERE is_canonical = 1")
global_canonicals = {row[1]: uuid.UUID(row[0]) for row in cursor.fetchall()}

print(f"Starting resolution with {len(entities)} entities. Found {len(global_canonicals)} canonicals")

# Group entities by type (e.g., 'Person', 'Organization') for more accurate matching
type_groups = defaultdict(list)
for entity in entities:
    type_groups[entity.type].append(entity)

resolved_id_map = {} # Maps an old entity ID to its new canonical ID
newly_created_canonicals = {}

for entity_type, group in type_groups.items():
    if not group: continue

    # Cluster entities in the group by fuzzy name matching
    clusters = []
    used_indices = set()
    for i in range(len(group)):
        if i in used_indices: continue
        current_cluster = [group[i]]
        used_indices.add(i)
        for j in range(i + 1, len(group)):
            if j in used_indices: continue
            # Use partial_ratio for flexible matching (e.g., "AMD" vs "Advanced")
            score = fuzz.partial_ratio(group[i].name.lower(), group[j].name.lower())
            if score > 0.8: # Threshold for clustering
                current_cluster.append(group[j])
                used_indices.add(j)
        clusters.append(current_cluster)

    # Resolve entities within each cluster
    for cluster in clusters:
        if len(cluster) == 1:
            entity = cluster[0]
            resolved_id_map[entity.id] = entity.name
            newly_created_canonicals[entity.name] = entity.id
        else:
            # Resolve cluster (e.g., find canonical name)
            canonical_name = cluster[0].name
            for entity in cluster:
                resolved_id_map[entity.id] = canonical_name
            newly_created_canonicals[canonical_name] = cluster[0].id
```

```

        used_indices.add(j)
        clusters.append(current_cluster)

# For each cluster, find the best canonical representation (the "medoid")
for cluster in clusters:
    scores = {e.name: sum(fuzz.ratio(e.name.lower(), other.name.lower())
                           for other in cluster) for e in cluster}
    medoid_entity = max(cluster, key=lambda e: scores[e.name])
    canonical_name = medoid_entity.name

    # Check if this canonical name already exists or was just created
    if canonical_name in global_canonicals:
        canonical_id = global_canonicals[canonical_name]
    elif canonical_name in newly_created_canonicals:
        canonical_id = newly_created_canonicals[canonical_name].id
    else:
        # Create a new canonical entity
        canonical_id = medoid_entity.id
        newly_created_canonicals[canonical_name] = medoid_entity

    # Map all entities in this cluster to the single canonical ID
    for entity in cluster:
        entity.resolved_id = canonical_id
        resolved_id_map[entity.id] = canonical_id

# Update the triplets in our state to use the new canonical IDs
for triplet in triplets:
    if triplet.subject_id in resolved_id_map:
        triplet.subject_id = resolved_id_map[triplet.subject_id]
    if triplet.object_id in resolved_id_map:

```

```
# Add any newly created canonical entities to our database
if newly_created_canonicals:
    print(f"Adding {len(newly_created_canonicals)} new canonical entities to database")
    new_data = [(str(e.id), e.name, e.type, e.description, 1) for e in newly_created_canonicals]
    cursor.executemany("INSERT INTO entities (id, name, type, description, count) VALUES (?, ?, ?, ?, ?)", new_data)
    db_conn.commit()

print("Entity resolution complete.")
return state
```

Now, let's add this new `resolve_entities` node to our LangGraph workflow. The new, improved flow will be: Start -> Extract -> Resolve -> End .

Copy

```
# Re-define the graph to include the new node
workflow = StateGraph(GraphState)

# Add our two nodes to the graph
workflow.add_node("extract_events", extract_events_from_chunks)
workflow.add_node("resolve_entities", resolve_entities_in_state)

# Define the new sequence of steps
workflow.set_entry_point("extract_events")
workflow.add_edge("extract_events", "resolve_entities")
workflow.add_edge("resolve_entities", END)
```

Let's run our new two-step pipeline on the same set of chunks and see the results.

Copy

```
# Use the same input as before
graph_input = {"chunks": chunked_documents_lc}

# Invoke the new graph
final_state_with_resolution = app_with_resolution.invoke(graph_input)

#### OUTPUT ####
--- Entering Node: extract_events_from_chunks ---
Processing 19 chunks...
Extracted a total of 95 statements from all chunks.
Completed batch extraction for dates and triplets.
Assembled 95 TemporalEvents, 213 Entities, and 121 Triplets.

--- Entering Node: resolve_entities_in_state ---
Starting resolution with 213 entities. Found 0 canonicals in DB.
Adding 110 new canonical entities to the DB.
Entity resolution complete.

--- Graph execution with resolution complete ---
```

canonical versions to our database.

Let's inspect the output to see the difference. We can check an entity to see if its `resolved_id` has been set, and check a triplet to confirm it's using the new, clean IDs.

Copy

```
# Find a sample entity that has been resolved (i.e., has a resolved_id)
sample_resolved_entity = next((e for e in final_state_with_resolution['entities']

if sample_resolved_entity:
    print("\n--- Sample of a Resolved Entity ---")
    print(sample_resolved_entity.model_dump_json(indent=2))
else:
    print("\nNo sample resolved entity found (all entities were unique in this

# Check a triplet to see its updated canonical IDs
sample_resolved_triplet = final_state_with_resolution['triplets'][0]
print("\n--- Sample Triplet with Resolved IDs ---")
print(sample_resolved_triplet.model_dump_json(indent=2))
```

This is what the resolved entity output is:


```
# --- Sample of a Resolved Entity ---
{
  "id": "1a2b3c4d-5e6f-4a7b-8c9d-0e1f2a3b4c5d",
  "name": "Advanced Micro Devices",
  "type": "Organization",
  "description": "A semiconductor company.",
  "resolved_id": "b1c2d3e4-f5g6-4h7i-8j9k-0l1m2n3o4p5q"
}

# --- Sample Triplet with Resolved IDs ---
{
  "id": "c1d2e3f4-a5b6-4c7d-8e9f-0g1h2i3j4k5l",
  "subject_name": "AMD",
  "subject_id": "b1c2d3e4-f5g6-4h7i-8j9k-0l1m2n3o4p5q",
  "predicate": "TARGETS",
  "object_name": "server launch",
  "object_id": "d1e2f3a4-b5c6-4d7e-8f9g-0h1i2j3k4l5m",
  "value": null
}
```

This is perfect. You can see that an entity named "Advanced Micro Devices" now has a `resolved_id` pointing to the ID of the canonical entity (which is likely the one named "AMD"). All our triplets now use these clean, canonical IDs.

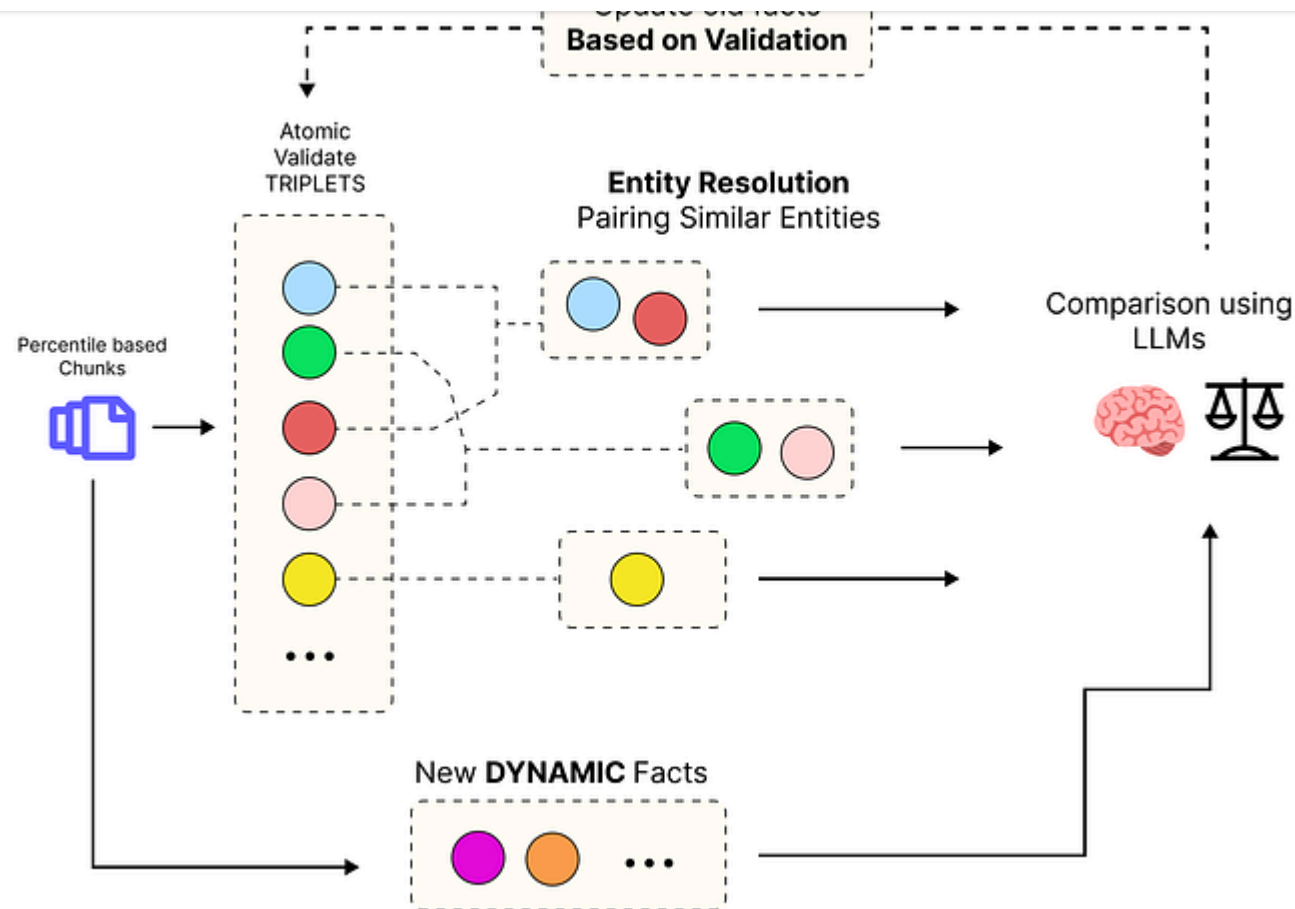
Our data is now not only structured and time-stamped, but also clean and consistent. The next step is the most intelligent part of our agent: handling

Making Our Knowledge Dynamic with an Invalidation Agent

Our data is now chunked, extracted, structured, and cleaned. But we still haven't addressed the core challenge of *temporal* data: facts change over time.

Imagine our knowledge base contains the fact: (John Smith) --[HOLDS_ROLE]--> (CFO) . This is a `DYNAMIC` fact, meaning it's true for a period of time. What happens when our agent reads a new document that says, "*Jane Doe was appointed CFO on January 1st, 2024*"?

The first fact is now outdated. A simple knowledge base wouldn't know this, but a *temporal* one must. This is the job of the **Invalidation Agent**: to act as a referee, find these contradictions, and update the old facts to mark them as expired.



Dynamic Knowledge Base (Created by [Fareed Khan](#))

To do this, we will add a final node to our LangGraph pipeline. This node will:

1. Generate embeddings for all our new statements to understand their semantic meaning.

3. Use an LLM to make a final judgment on whether a new fact invalidates an old one.

4. If a fact is invalidated, it will update its `invalid_at` timestamp.

First, we need to prepare our environment for the invalidation logic. This involves adding the `events` and `triplets` tables to our in-memory database. This simulates a real, persistent knowledge base that the agent can check against.

Copy

```
# Obtain a cursor from the existing database connection
cursor = db_conn.cursor()

# Create the 'events' table to store event-related data
cursor.execute("""
CREATE TABLE IF NOT EXISTS events (
    id TEXT PRIMARY KEY,          -- Unique identifier for each event
    chunk_id TEXT,                -- Identifier for the chunk this event belongs to
    statement TEXT,               -- Textual representation of the event
    statement_type TEXT,          -- Type/category of the statement (e.g., assertion, question)
    temporal_type TEXT,           -- Temporal classification (e.g., past, present, future)
    valid_at TEXT,                -- Timestamp when the event becomes valid
    invalid_at TEXT,              -- Timestamp when the event becomes invalid
    embedding BLOB                -- Optional embedding data stored as binary (e.g., vector)
)
""")
```

```
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS triplets (
        id TEXT PRIMARY KEY,          -- Unique identifier for each triplet
        event_id TEXT,                -- Foreign key referencing 'events.id'
        subject_id TEXT,              -- Subject entity ID in the triplet
        predicate TEXT                -- Predicate describing relation or action
    )
    """
)

# Commit all changes to the in-memory database
db_conn.commit()
```

ext, we'll create a new prompt and chain. This chain is very simple: it will show the LLM two potentially conflicting events and ask for a simple "True" or "False" decision on whether one invalidates the other.

Copy

```
# This prompt asks the LLM to act as a referee between two events.
event_invalidation_prompt_template = """
Task: Analyze the primary event against the secondary event and determine if the
primary event is invalidated by the secondary event. Return "True" if the primary event is
invalidated, otherwise return "False".

Invalidation Guidelines:
1. An event can only be invalidated if it is DYNAMIC and its `invalid_at` is current or
past.
2. A STATIC event (e.g., "X was hired on date Y") can invalidate a DYNAMIC event.
3. Invalidation must be a direct contradiction. For example, "Lisa Su is CEO" :
"""
```

```
---
Primary Event (the one that might be invalidated):
- Statement: {primary_statement}
- Type: {primary_temporal_type}
- Valid From: {primary_valid_at}
- Valid To: {primary_invalid_at}

Secondary Event (the new fact that might cause invalidation):
- Statement: {secondary_statement}
- Type: {secondary_temporal_type}
- Valid From: {secondary_valid_at}
---

Is the primary event invalidated by the secondary event? Answer with only "True"
or "False".

invalidation_prompt = ChatPromptTemplate.from_template(event_invalidation_prompt)

# This chain will output a simple string: "True" or "False".
invalidation_chain = invalidation_prompt | llm
```

Now we'll write the function for our new LangGraph node. This is the most complex function in our pipeline, but it follows a clear logic: find potentially related facts using embeddings, and then ask the LLM to make the final call.

Copy

```

def invalidate_events_in_state(state: GraphState) -> GraphState:
    """Mark dynamic events invalidated by later similar facts."""
    events = state["temporal_events"]

    # Embed all event statements
    embeds = embeddings.embed_documents([e.statement for e in events])
    for e, emb in zip(events, embeds):
        e.embedding = emb

    updates = {}
    for i, e1 in enumerate(events):
        # Skip non-dynamic or already invalidated events
        if e1.temporal_type != TemporalType.DYNAMIC or e1.invalid_at:
            continue

        # Find candidate events: facts starting at or after e1 with high similarity
        candids = [
            e2 for j, e2 in enumerate(events) if j != i and
            e2.statement_type == StatementType.FACT and e2.valid_at and e1.valid_at and
            e2.valid_at >= e1.valid_at and 1 - cosine(e1.embedding, e2.embedding) > SIMILARITY_THRESHOLD
        ]
        if not candids:
            continue

        # Prepare inputs for LLM invalidation check
        inputs = [{
            "primary_statement": e1.statement, "primary_temporal_type": e1.temporal_type,
            "primary_valid_at": e1.valid_at.isoformat(), "primary_invalid_at": e1.invalid_at.isoformat(),
            "secondary_statement": c.statement, "secondary_temporal_type": c.temporal_type,
            "secondary_valid_at": c.valid_at.isoformat(), "secondary_invalid_at": c.invalid_at.isoformat()
        } for c in candids]
        results = llm.invoke(inputs)
        for i, result in zip(candids, results):
            if result == "INVALID":
                candids[i].invalid_at = datetime.now()
            else:
                candids[i].valid_at = datetime.now()

    for e in events:
        if e in candids:
            e.embedding = embeddings.embed_documents([e.statement])[0]

    return state.update(updates)

```

```
# Ask LLM which candidates invalidate the event
results = invalidation_chain.batch(inputs)

# Record earliest invalidation info
for c, r in zip(cands, results):
    if r.content.strip().lower() == "true" and (e1.id not in updates or
        updates[e1.id] = {"invalid_at": c.valid_at, "invalidated_by": c

# Apply invalidations to events
for e in events:
    if e.id in updates:
        e.invalid_at = updates[e.id]["invalid_at"]
        e.invalidated_by = updates[e.id]["invalidated_by"]

return state
```

Now, let's add our final `invalidate_events` node to the LangGraph workflow. The completed ingestion pipeline will be: **Extract -> Resolve -> Invalidate**.

Copy

```
# Re-define the graph to include all three nodes
workflow = StateGraph(GraphState)

workflow.add_node("extract_events", extract_events_from_chunks)
workflow.add_node("resolve_entities", resolve_entities_in_state)
```



```
# DEFINE THE COMPLETE PIPELINE FLOW
workflow.set_entry_point("extract_events")
workflow.add_edge("extract_events", "resolve_entities")
workflow.add_edge("resolve_entities", "invalidate_events")
workflow.add_edge("invalidate_events", END)

# Compile the final ingestion workflow
ingestion_app = workflow.compile()
```

Now, let's run the full, three-step pipeline on our sample document's chunks.

Copy

```
# Use the same input as before
graph_input = {"chunks": chunked_documents_lc}

# Invoke the final graph
final_ingested_state = ingestion_app.invoke(graph_input)
print("\n--- Full graph execution with invalidation complete ---")

#### OUTPUT ####
--- Entering Node: extract_events_from_chunks ---
Processing 19 chunks...
Extracted a total of 95 statements from all chunks.
...
--- Entering Node: resolve_entities_in_state ---
```

```

--- Entering Node: Inactivate_events_in_state ---
Generated embeddings for 95 events.
...
Checking for invalidations: 100%|██████████| 95/95 [00:08<00:00, 11.23it/s]
Found 1 invalidations to apply.

--- Full graph execution with invalidation complete ---

```

The pipeline ran successfully and even found an invalidation to apply! Let's find that specific event to see what happened.

Copy

```

# Find and print an invalidated event from the final state
invalidated_event = next((e for e in final_ingested_state['temporal_events'] if

if invalidated_event:
    print("\n--- Sample of an Invalidated Event ---")
    print(invalidated_event.model_dump_json(indent=2))

# Find the event that caused the invalidation
invalidating_event = next((e for e in final_ingested_state['temporal_events']

if invalidating_event:
    print("\n--- Was Invalidated By this Event ---")
    print(invalidating_event.model_dump_json(indent=2))

```

This is what the invalidation event occurs in our chunk data.

Copy

```
# --- Sample of an Invalidated Event ---
{
  "id": "e5094890-7679-4d38-8d3b-905c11b0ed08",
  "statement": "All participants are in a listen-only mode...",
  "statement_type": "FACT",
  "temporal_type": "DYNAMIC",
  "valid_at": "2016-07-21T00:00:00+00:00",
  "invalid_at": "2016-07-21T00:00:00+00:00",
  "invalidated_by": "971ffb90-b973-4f41-a718-737d6d2e0e38"
}

# --- Was Invalidated By this Event ---
{
  "id": "971ffb90-b973-4f41-a718-737d6d2e0e38",
  "statement": "The Q&A session will begin now.",
  "statement_type": "FACT",
  "temporal_type": "STATIC",
  "valid_at": "2016-07-21T00:00:00+00:00",
  "invalid_at": null,
  "invalidated_by": null
}
```

begin now". It updated the first event, setting its `invalid_at` timestamp.

This completes the entire ingestion pipeline. We have built a system that can automatically create a clean, structured, and temporally-aware knowledge base from raw text. The next and final step is to take this rich data and build our knowledge graph.

Assembling the Temporal Knowledge Graph

So far, we have successfully takes raw, messy transcripts and transforms them into a clean, structured, and temporally-aware collection of facts.

We have successfully automated:

1. **Extraction:** Pulling out statements, dates, and triplets.
2. **Resolution:** Cleaning up duplicate entities.
3. **Invalidation:** Updating facts that are no longer true.

Now, it's time to take this final, high-quality data and build our knowledge graph. This graph will be the "brain" that our retrieval agent will query to answer user questions.

- **Entities** (like "AMD" or "Lisa Su") become the **nodes** (the dots).
- **Triples** (the relationships) become the **edges** (the lines connecting the dots).

This structure allows us to easily traverse from one fact to another, which is exactly what a smart retrieval agent needs to do.

we will use a popular Python library called `NetworkX` to build our graph. It's lightweight, easy to use, and perfect for working with graph structures directly in our notebook.

we'll write a function that takes the final state from our `LangGraph` pipeline (`final_ingested_state`) and constructs a `NetworkX` graph object.

This function will create a node for each unique, canonical entity and an edge for each triplet, adding all of our rich temporal data as attributes on that edge.

Copy

```
import networkx as nx
import uuid

def build_graph_from_state(state: GraphState) -> nx.MultiDiGraph:
```

```
print("--- Building Knowledge Graph from final state ---")

entities = state["entities"]
triplets = state["triplets"]
temporal_events = state["temporal_events"]

# Create a quick-lookup map from an entity's ID to the entity object itself
entity_map = {entity.id: entity for entity in entities}

graph = nx.MultiDiGraph() # A directed graph that allows multiple edges

# 1. Add a node for each unique, canonical entity
canonical_ids = {e.resolved_id if e.resolved_id else e.id for e in entities}
for canonical_id in canonical_ids:
    # Find the entity object that represents this canonical ID
    canonical_entity_obj = entity_map.get(canonical_id)
    if canonical_entity_obj:
        graph.add_node(
            str(canonical_id), # Node names in NetworkX are typically strings
            name=canonical_entity_obj.name,
            type=canonical_entity_obj.type,
            description=canonical_entity_obj.description
        )

print(f"Added {graph.number_of_nodes()} canonical entity nodes to the graph")

# 2. Add an edge for each triplet, decorated with temporal info
edges_added = 0
```

```
# Find the parent event that this triplet belongs to
parent_event = next((ev for ev in temporal_events if triplet.id in ev.id), None)
if not parent_event: continue

# Get the canonical IDs for the subject and object
subject_canonical_id = str(triplet.subject_id)
object_canonical_id = str(triplet.object_id)

# Add the edge to the graph
if graph.has_node(subject_canonical_id) and graph.has_node(object_canonical_id):
    edge_attrs = {
        "predicate": triplet.predicate.value, "value": triplet.value,
        "statement": parent_event.statement, "valid_at": parent_event.valid_at,
        "invalid_at": parent_event.invalid_at,
        "statement_type": parent_event.statement_type.value
    }
    graph.add_edge(
        subject_canonical_id, object_canonical_id,
        key=triplet.predicate.value, **edge_attrs
    )
    edges_added += 1

print(f"Added {edges_added} edges (relationships) to the graph.")
return graph

# Let's build the graph from the state we got from our LangGraph app
knowledge_graph = build_graph_from_state(final_ingested_state)
```

Copy

```
# --- Building Knowledge Graph from final state ---  
Added 340 canonical entity nodes to the graph.  
Added 434 edges (relationships) to the graph.
```

So, we have total 340 entities and 434 edges on our first chunk data. Our function has successfully converted the lists of objects from our pipeline into a formal graph structure.

Now that we have our `knowledge_graph` object, let's explore it to see what we've built. We can inspect a specific node, like "AMD", to see its properties and relationships.

Copy

```
print(f"Graph has {knowledge_graph.number_of_nodes()} nodes and {knowledge_graph.number_of_edges()} edges")  
  
# Let's find the node for "AMD" by searching its 'name' attribute  
amd_node_id = None  
for node, data in knowledge_graph.nodes(data=True):  
    if data.get('name', '').lower() == 'amd':  
        amd_node_id = node  
        break
```



```
print(f"Attributes: {knowledge_graph.nodes[amd_node_id]}")

print("\n--- Sample Outgoing Edges from 'AMD' ---")
for i, (u, v, data) in enumerate(knowledge_graph.out_edges(amd_node_id, data=True)):
    if i >= 3: break # Show the first 3 for brevity
    object_name = knowledge_graph.nodes[v]['name']
    print(f"Edge {i+1}: AMD --[{data['predicate']}]--> {object_name} (Valid From: {data['valid_from']})")
else:
    print("Could not find a node for 'AMD'.")
```

This is the output.

Copy

Graph has 340 nodes and 434 edges.

--- Inspecting the 'AMD' node ---

Attributes: {'name': 'AMD', 'type': 'Organization', 'description': ''}

--- Sample Outgoing Edges from 'AMD' ---

Edge 1: AMD --[HOLDS_ROLE]--> President and CEO (Valid From: 2016-07-21)

Edge 2: AMD --[HAS_A]--> SVP, CFO, and Treasurer (Valid From: 2016-07-21)

Edge 3: AMD --[HAS_A]--> Chief Human Resources Officer and SVP of Corporate Communication

We can see the node for AMD and some of the relationships connected to it, along with the date that relationship became valid.

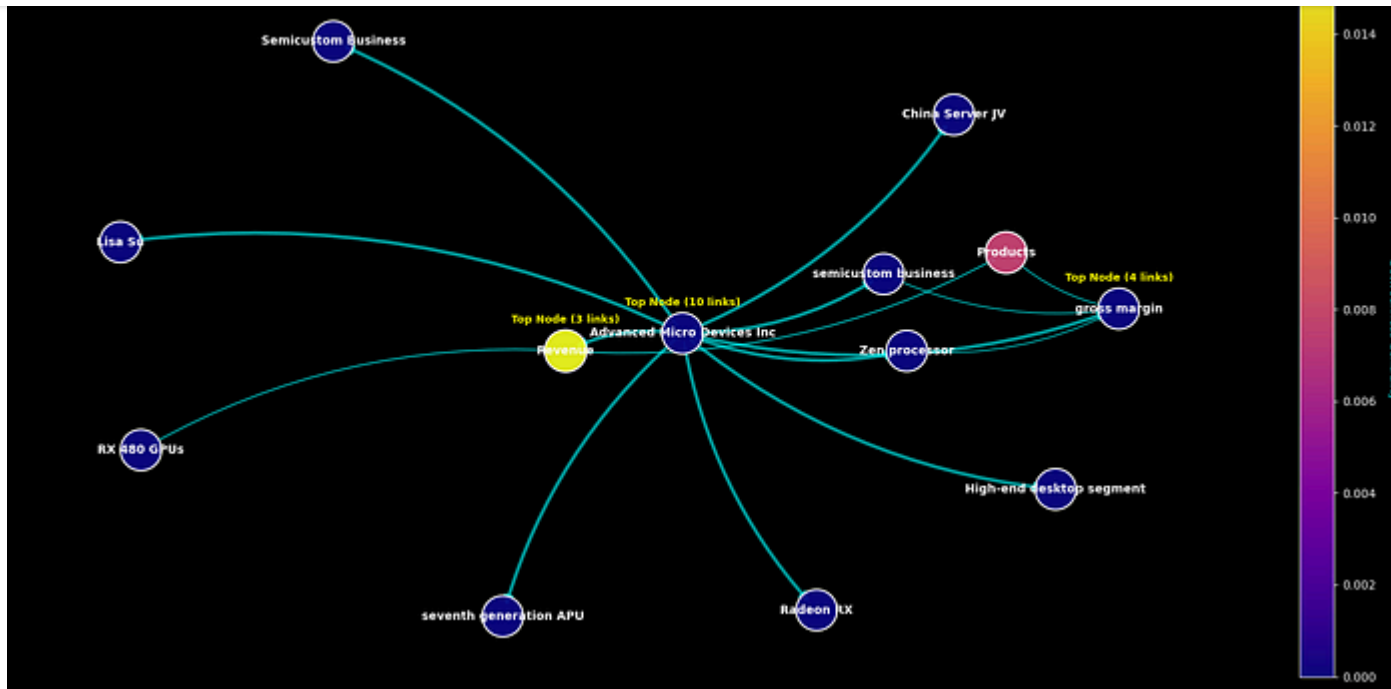
Copy

```
import matplotlib.pyplot as plt

# Find the 15 most connected nodes to visualize
degrees = dict(knowledge_graph.degree())
top_nodes = sorted(degrees, key=degrees.get, reverse=True)[:15]

# Create a smaller graph containing only these top nodes
subgraph = knowledge_graph.subgraph(top_nodes)

# Draw the graph
plt.figure(figsize=(12, 12))
pos = nx.spring_layout(subgraph, k=0.8, iterations=50)
labels = {node: data['name'] for node, data in subgraph.nodes(data=True)}
nx.draw(subgraph, pos, labels=labels, with_labels=True, node_color='skyblue',
        node_size=2500, edge_color='#666666', font_size=10)
plt.title("Subgraph of Top 15 Most Connected Entities", size=16)
plt.show()
```



Sample Knowledge Graph (Created by [Fareed Khan](#))

This visualization gives us a bird's-eye view of the core entities and relationships we've extracted. We have successfully created our Temporal Knowledge Graph.

We can now build an smart agent that can talk to this advanced temporal knowledge graph database to answer complex, time-sensitive questions.

Building and Testing A Multi-Step Retrieval Agent

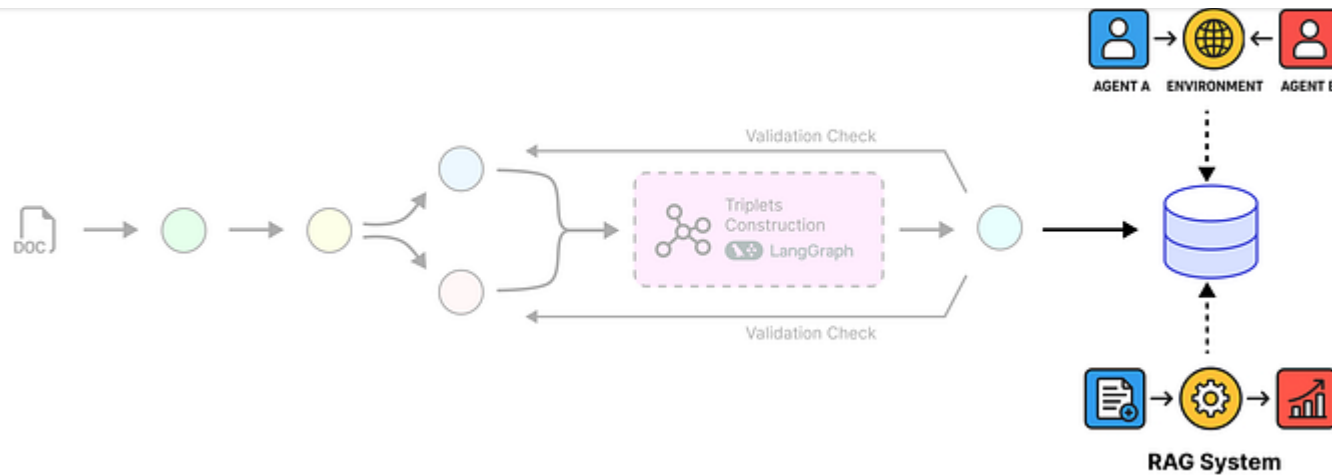
have a conversation with this graph to answer complex questions.

Why Single-Step RAG Isn't Enough

A simple RAG system might find one relevant fact and use it to answer a question. But what if the answer requires connecting multiple pieces of information?

Consider a question like ...

How did AMD's focus on data centers evolve between 2016 and 2017?

Multi Agent System (Created by [Fareed Khan](#))

A simple retrieval system can't answer this. It requires a **multi-step** process:

1. Find facts about AMD and data centers in 2016.
2. Find facts about AMD and data centers in 2017.
3. Compare the results from both years.
4. Synthesize a final summary.

This is what our multi-step retrieval agent will do. We will build it using LangGraph, and it will have three key components: a Planner, a set of Tools, and an Orchestrator.

LLM chain whose only job is to break down the user's question into a sequence of actionable steps.

Let's create the prompt for our planner. We'll give it a "persona" as an expert financial research assistant to guide its tone and the quality of the plan it creates.

Copy

```
# System prompt describes the "persona" for the LLM
initial_planner_system_prompt = (
    "You are an expert financial research assistant. "
    "Your task is to create a step-by-step plan for answering a user's question "
    "by querying a temporal knowledge graph of earnings call transcripts. "
    "The available tool is `factual_qa`, which can retrieve facts about an entity "
    "for a specific topic (predicate) within a given date range. "
    "Your plan should consist of a series of calls to this tool."
)

# Template for the user prompt – receives `user_question` dynamically
initial_planner_user_prompt_template = """
User Question: "{user_question}"

Based on this question, create a concise, step-by-step plan.
Each step should be a clear action for querying the knowledge graph.

Return only the plan under a heading 'Research tasks'.
```

```
# Create a ChatPromptTemplate that combines the system persona and the user prompt
# `from_messages` takes a list of (role, content) pairs to form the conversation
planner_prompt = ChatPromptTemplate.from_messages([
    ("system", initial_planner_system_prompt),      # LLM's role and behavior
    ("user", initial_planner_user_prompt_template), # Instructions for this task
])

# Create a "chain" that pipes the prompt into the LLM.
# The `|` operator here is the LangChain "Runnable" syntax for composing components
planner_chain = planner_prompt | llm
```

Now, let's test the planner with our sample question to see what kind of strategy it develops.

Copy

```
# Our sample user question for the retrieval agent
user_question = "How did AMD's focus on data centers evolve between 2016 and 2020?"

print(f"--- Generating plan for question: '{user_question}' ---")
plan_result = planner_chain.invoke({"user_question": user_question})
initial_plan = plan_result.content

print("\n--- Generated Plan ---")
print(initial_plan)

#### OUTPUT ####
```

```
--- Generated Plan ---
```

Research tasks:

1. Query the `factual\_qa` tool for the entity "AMD" with predicates related to
2. Query the `factual\_qa` tool for the entity "AMD" with the same predicates t
3. Synthesize the results from both queries to describe the evolution of AMD's

Based on the output of generated plan we can see that it gives our main agent a clear roadmap to follow. It knows exactly what information it needs to find to answer the user's question.

Copy

```
from langchain_core.tools import tool
from datetime import date
import datetime as dt # Use an alias to avoid confusion

# Helper function to parse dates robustly, even if the LLM provides different t
def _as_datetime(ts) -> dt.datetime | None:
    if not ts: return None
    if isinstance(ts, dt.datetime): return ts
    if isinstance(ts, dt.date): return dt.datetime.combine(ts, dt.datetime.min.time())
    try:
        return dt.datetime.strptime(ts, "%Y-%m-%d")
    except (ValueError, TypeError):
        return None

@tool
```



```

queries the knowledge graph for facts about a specific entity, topic (pred.)
and time range. Returns a formatted string of matching relationships.
"""

print(f"\n--- TOOL CALL: factual_qa ---")
print(f" - Entity: {entity}, Predicate: {predicate}, Range: {start_date} to {end_date}")

start_dt = _as_datetime(start_date).replace(tzinfo=timezone.utc)
end_dt = _as_datetime(end_date).replace(tzinfo=timezone.utc)

# 1. Find the entity node in the graph using a case-insensitive search
target_node_id = next((nid for nid, data in knowledge_graph.nodes(data=True)
                        if data.get('name', '').lower() == entity.lower()), None)
if not target_node_id: return f"Error: Entity '{entity}' not found."

# 2. Search all edges connected to that node for matches
matching_edges = []
for u, v, data in knowledge_graph.edges(target_node_id, data=True):
    if predicate.upper() in data.get('predicate', '').upper():
        valid_at = data.get('valid_at')
        if valid_at and start_dt <= valid_at <= end_dt:
            subject = knowledge_graph.nodes[u]['name']
            obj = knowledge_graph.nodes[v]['name']
            matching_edges.append(f"Fact: {subject} --[{data['predicate']}]-- {obj}")

if not matching_edges: return f"No facts found for '{entity}' with predicate '{predicate}'"
return "\n".join(matching_edges)

```

This `factual_qa` function is the bridge between our agent's brain (the LLM) and its memory (the knowledge graph).

call in a loop until it has enough information to provide a final answer.

We will build this orchestrator using LangGraph, which is perfect for managing this kind of cyclical, **"decide-and-act"** logic.

Copy

```
from langgraph.prebuilt import ToolNode
from langgraph.graph import StateGraph, END
from langchain_core.messages import BaseMessage, HumanMessage
from typing import TypedDict, List

# Define the state for our retrieval agent's memory
class AgentState(TypedDict):
    messages: List[BaseMessage]

# This is the "brain" of our agent. It decides what to do next.
def call_model(state: AgentState):
    print("\n--- AGENT: Calling model to decide next step... ---")
    response = llm_with_tools.invoke(state['messages'])
    return {"messages": [response]}

# This is a conditional edge. It checks if the LLM decided to call a tool or to
def should_continue(state: AgentState) -> str:
    if hasattr(state['messages'][-1], 'tool_calls') and state['messages'][-1].tool_calls:
        return "continue_with_tool"
    return "finish"
```

```
# This is required by our specific model
tools = [factual_qa]
llm_with_tools = llm.bind_tools(tools, tool_choice="any")

# Now, wire up the graph
workflow = StateGraph(AgentState)
workflow.add_node("agent", call_model)
workflow.add_node("action", ToolNode(tools)) # ToolNode is a pre-built node that
workflow.set_entry_point("agent")
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {"continue_with_tool": "action", "finish": END}
)
workflow.add_edge("action", "agent")

retrieval_agent = workflow.compile()
```

Our retrieval agent is now fully built! It has a brain (`call_model`), hands (`ToolNode`), and a looping mechanism to allow it to think and act multiple times.

Let's run the full process and ask our question. We'll give the agent the user question *and* the plan we generated earlier to give it the best possible start.

Copy

```
        content=f"Here is my question: '{user_question}'\n\n"
        f"Here is the plan to follow:\n{initial_plan}"
    )

    # The input to the agent is always a list of messages
    agent_input = {"messages": [initial_message]}

    print("--- Running the full retrieval agent ---")

    # Stream the agent's execution to see its thought process in real-time
    async for output in retrieval_agent.astream(agent_input):
        for key, value in output.items():
            if key == "agent":
                agent_message = value['messages'][-1]
                if agent_message.tool_calls:
                    print(f"LLM wants to call a tool: {agent_message.tool_calls[0]}")
                else:
                    print("\n--- AGENT: Final Answer ---")
                    print(agent_message.content)
            elif key == "action":
                print("--- AGENT: Tool response received. ---")
```

Now, when we run the full retireval agent using the asyn approach, this is what the response looks like.

Copy

```
# --- AGENT: Calling model to decide next step... ---
LLM wants to call a tool: factual_qa

# --- TOOL CALL: factual_qa ---
- Entity: AMD, Predicate: LAUNCHED, Range: 2016-01-01 to 2016-12-31
--- AGENT: Tool response received. ---

# --- AGENT: Calling model to decide next step... ---
LLM wants to call a tool: factual_qa

# --- TOOL CALL: factual_qa ---
- Entity: AMD, Predicate: LAUNCHED, Range: 2017-01-01 to 2017-12-31
--- AGENT: Tool response received. ---

# --- AGENT: Calling model to decide next step... ---

# --- AGENT: Final Answer ---
Based on the information from the knowledge graph, here is how AMDs focus on data
* **2016:** There were no facts found related to AMD launching products for c
* **2017:** A key development occurred. The knowledge graph contains a fact s

This indicates a significant shift. While 2016 showed no specific launch activ
```

The agent successfully followed the plan, called its tool twice with different date ranges, and then synthesized the results into a perfect, comparative answer. This

What Can We Do Next?

We have built a powerful prototype that demonstrates how to create a knowledge base that isn't just a static library, but a dynamic system that understands how facts evolve over time.

Now that our agent works, the next critical question is: "How *well* does it work?" Answering this requires a formal evaluation process. There are three main approaches:

- **Golden Answers (The Gold Standard):** You create a test set of questions and have human experts write the perfect answers. You then compare your agent's output to these "golden" answers. This is the most accurate method but is slow and expensive.
- **LLM-as-Judge (The Scalable Approach):** You use a powerful LLM (like GPT-4) to act as a "judge." It scores your agent's answers for correctness and relevance. This is fast and cheap, making it perfect for rapid testing and iteration.
- **Human Feedback (The Real-World Test):** Once your agent is deployed, you can add simple feedback buttons (like thumbs-up/thumbs-down) to let users

Freedium[ko-fi](#) [librepatron](#) [patreon](#)

In case you enjoy this blog, feel free to [follow me on Medium](#) I only write here.

[#ai](#) [#artificial-intelligence](#) [#data-science](#) [#python](#) [#machine-learning](#)